Chapter 21 — Interview Question Bank

How to use this bank

This chapter collects, in one list, the questions that SQL and database interviews keep asking, worded the way they usually appear on LeetCode, in interview-experience posts and in the big public question lists. It has two parts:

- **Part A — Concept questions (Q1–Q120).** "Explain X", "X vs Y", "what does this query return" and scenario questions on keys, joins, NULL logic, normalization, indexes and `EXPLAIN`, transactions, isolation and locking, views, CTEs, window functions, procedures and triggers, SQL injection, partitioning, replication, sharding and SQL vs NoSQL.
- **Part B — Coding problems (Q121–Q250).** Every problem of LeetCode's **SQL 50** study plan, in the plan's order, followed by the other database problems that are asked most often (Nth Highest Salary, Rank Scores, Trips and Users, Human Traffic of Stadium, the medians, the Game Play Analysis series and more). Each is listed by its LeetCode title and number.

How to read an entry:

- ⋆ marks a question that appears in many sources and is asked very often. Learn those first.
- Each concept answer is short enough to say out loud in 30–60 seconds. The *More* link points to the full explanation in this book.
- Each coding entry gives the approach in 1–3 sentences and a PostgreSQL solution. When the book already solves the same problem (Chapters 18–19 restate it with their own table names and data), the entry gives the approach and a link instead of repeating the solution.
- All SQL is PostgreSQL and was run on a real server. Features newer than PostgreSQL 14 are marked (15+, 16+, …).

LeetCode's PostgreSQL judge. LeetCode creates its tables with unquoted names, so `Employee` and `managerId` are stored as `employee` and `managerid`, and you can type them in any case. When the expected output has a mixed-case column name such as `SecondHighestSalary`, the solutions quote the alias (`AS "SecondHighestSalary"`) so the case is kept. Most rounding questions need `round(x::numeric, 2)` in PostgreSQL, because `round(double precision, int)` does not exist.

How to drill. Cover the answer, say yours out loud (or write the query on paper), then check. For coding problems, first say the pattern (anti-join, top-N per group, gaps and islands, conditional aggregation…), then the query. Mark the ones you missed and repeat them two days later.

Part A — Concept questions

Fundamentals and keys

Q1. ⋆ What is the difference between a database, a DBMS and SQL?

A **database** is an organised, persistent collection of related data. A **DBMS** (database management system) is the software that stores it, answers queries, enforces rules, controls concurrent access and recovers after crashes; PostgreSQL, MySQL and Oracle are DBMSs. **SQL** (Structured Query Language) is the declarative language you use to talk to a relational DBMS: you say *what* you want, and the DBMS decides *how* to get it. → *More:* Ch 1 — What a DBMS does

Q2. What are DDL, DML, DQL, DCL and TCL? Give examples.

They are SQL's sub-languages. **DDL** (data definition) changes structure: `CREATE`, `ALTER`, `DROP`, `TRUNCATE`. **DML** (data manipulation) changes rows: `INSERT`, `UPDATE`, `DELETE`, `MERGE`. **DQL** (data query) reads: `SELECT`. **DCL** (data control) manages permissions: `GRANT`, `REVOKE`. **TCL** (transaction control) groups work: `BEGIN`, `COMMIT`, `ROLLBACK`, `SAVEPOINT`. In PostgreSQL even DDL is transactional, so a `CREATE TABLE` can be rolled back. → *More:* Ch 1 — What SQL is

Q3. ⋆ What is a relational database? Define table, row, column, relation and tuple.

A relational database stores data in **relations** (tables). Each **row** (tuple) is one fact, each **column** (attribute) has a name and a data type, and rows are identified by keys rather than by their position. Tables are connected through matching key values, not pointers, so any question can be answered by combining tables with joins. → *More:* Ch 1 — The relational model

Q4. ⋆ Primary key vs unique key vs foreign key?

A **primary key** (PK) is the identifier chosen for the table: unique, never NULL, one per table. A **unique key** (unique constraint) also forbids duplicates but is an alternative identifier, and it allows NULLs (in PostgreSQL several NULLs, unless you declare `NULLS NOT DISTINCT`, 15+). A **foreign key** (FK) is a column whose values must exist in another table's PK or unique key; it enforces **referential integrity**, which means no row points at a parent that does not exist. → *More:* Ch 9 — Keys and constraints

Q5. Superkey vs candidate key vs alternate key vs composite key?

A **superkey** is any set of columns that identifies a row uniquely. A **candidate key** is a minimal superkey: remove any column and it stops being unique. The candidate key you pick becomes the primary key, and the others are **alternate keys**. A **composite key** is any key made of two or more columns, such as `(order_id, line_no)`. → *More:* Ch 10 — Keys in relational databases

Q6. ⋆ Natural key vs surrogate key — which would you use?

A **natural key** comes from the business data (an e-mail, an ISBN). A **surrogate key** is a generated value with no meaning (`bigint GENERATED ALWAYS AS IDENTITY` or a UUID). Natural keys can change and are often long; surrogate keys never change and keep foreign keys small. The usual design is a surrogate PK plus a `UNIQUE` constraint on the natural key, so you get both stability and the business rule. → *More:* Ch 9 — Generated keys

Q7. What constraints does SQL offer?

`NOT NULL` (a value is required), `UNIQUE`, `PRIMARY KEY` (unique + not null), `FOREIGN KEY` (value must exist in the parent), `CHECK` (a boolean rule per row, such as `price >= 0`) and `DEFAULT` (value used when none is given). PostgreSQL adds `EXCLUDE` constraints, which forbid overlapping rows such as two bookings of the same room at overlapping times. Constraints are the last line of defence: application checks can be bypassed, constraints cannot. → *More:* Ch 9 — Keys and constraints

Q8. What do `ON DELETE CASCADE`, `SET NULL` and `RESTRICT` do on a foreign key?

They say what happens to child rows when the parent row is deleted (or its key updated). `CASCADE` deletes (or updates) the children too, `SET NULL` / `SET DEFAULT` clear the reference, and `NO ACTION` (the default) and `RESTRICT` refuse the delete while children exist (`RESTRICT` checks at once; `NO ACTION` checks at the end of the statement and can be deferred to commit). Use `CASCADE` only for true ownership (order → order lines), never for shared reference data. → *More:* Ch 9 — Keys and constraints

Q9. ⋆ `DELETE` vs `TRUNCATE` vs `DROP`?

`DELETE` removes the rows that match a `WHERE`, row by row, fires row triggers and can be filtered. `TRUNCATE` empties the whole table at once by replacing its files, which is much faster, does not fire `DELETE` triggers (only `TRUNCATE` triggers) and can reset identity columns. `DROP TABLE` removes the table itself (structure, indexes, privileges). In PostgreSQL all three are transactional and can be rolled back; in MySQL and Oracle `TRUNCATE` commits implicitly. → *More:* Ch 9 — DROP and TRUNCATE

Q10. `CHAR` vs `VARCHAR` vs `TEXT` in PostgreSQL?

`char(n)` pads with spaces to length n, `varchar(n)` stores up to n characters, and `text` has no limit. In PostgreSQL they are stored the same way and perform the same, so `text` (or `varchar(n)` when a length limit is a real business rule) is the usual choice and `char(n)` is almost never right. In other databases the trade-offs differ (for example index key size limits in MySQL). → *More:* Ch 9 — Data types

Q11. Why should money be stored as `numeric` and not `float`?

`float` / `double precision` are binary approximations, so `0.1 + 0.2` is not exactly `0.3` and sums drift. `numeric(p, s)` stores exact decimal digits, which is what accounting needs. A common alternative is an integer number of cents. → *More:* Ch 3 — Data types and conversions

Q12. How do you implement an "upsert" (insert, or update if it exists)?

In PostgreSQL use `INSERT … ON CONFLICT (key) DO UPDATE SET col = EXCLUDED.col`, where `EXCLUDED` is the row you tried to insert; it needs a unique index on the conflict columns and is safe under concurrency. `ON`

`CONFLICT DO NOTHING` skips duplicates. PostgreSQL 15+ also has the standard `MERGE`, which is more flexible but, unlike `ON CONFLICT`, can still fail with a unique violation under concurrent inserts.

```
INSERT INTO stock (sku, qty) VALUES ('A1', 5)
ON CONFLICT (sku) DO UPDATE SET qty = stock.qty + EXCLUDED.qty;
```

→ *More:* Ch 9 — Upsert

SELECT, NULL logic and query order

Q13. ⋆ In what order is a `SELECT` statement logically evaluated?

`FROM` / `JOIN` → `WHERE` → `GROUP BY` → aggregates → `HAVING` → window functions → `SELECT` list → `DISTINCT` → `UNION` / `INTERSECT` / `EXCEPT` → `ORDER BY` → `LIMIT` / `OFFSET`. This explains many errors: a column alias from `SELECT` cannot be used in `WHERE` (it does not exist yet), but it can be used in `ORDER BY`. The planner may execute in a different physical order as long as the result is the same. → *More:* Ch 4 — The logical order of query processing

Q14. ⋆ What is NULL, and why does `WHERE col = NULL` return nothing?

NULL means "unknown or missing", not zero or empty string. Any comparison with NULL gives **UNKNOWN**, the third truth value, and `WHERE` keeps only rows where the condition is TRUE. So test with `IS NULL` / `IS NOT NULL`, or use `IS DISTINCT FROM` for a NULL-safe "not equal". → *More:* Ch 2 — NULL and three-valued logic

Q15. ⋆ What does this return: `SELECT NULL = NULL, NULL AND FALSE, NULL OR TRUE, 1 + NULL;`?

`NULL` (unknown), `false`, `true` and `NULL`. `FALSE AND anything` is FALSE and `TRUE OR anything` is TRUE, because the unknown value cannot change the outcome; arithmetic with NULL is NULL.

Q16. ⋆ Why can `NOT IN (subquery)` return no rows at all?

`x NOT IN (1, 2, NULL)` means `x <> 1 AND x <> 2 AND x <> NULL`. The last part is UNKNOWN, so the whole condition is never TRUE and every row is dropped. Use `NOT EXISTS`, which only asks whether a matching row exists and is not affected by NULLs. → *More:* Ch 6 — Multi-row subqueries: IN and NOT IN

Q17. `COALESCE` vs `NULLIF` vs `CASE`?

`COALESCE(a, b, c)` returns the first non-NULL argument (a default for missing values). `NULLIF(a, b)` returns NULL when `a = b`, otherwise `a`; the classic use is `x / NULLIF(y, 0)` to avoid division by zero. `CASE WHEN … THEN … ELSE … END` is the general if/else, and both functions are shorthands for a `CASE`. → *More:* Ch 3 — Conditional expressions

Q18. ⋆ `WHERE` vs `HAVING`?

`WHERE` filters individual rows **before** grouping and cannot use aggregate functions. `HAVING` filters **groups** after `GROUP BY` and can use aggregates such as `HAVING count(*) > 5`. Put row conditions in `WHERE`: it removes rows early, which is faster. → *More:* Ch 4 — HAVING

Q19. ⋆ `COUNT(*)` vs `COUNT(col)` vs `COUNT(DISTINCT col)`?

`count(*)` counts rows. `count(col)` counts rows where `col` is not NULL. `count(DISTINCT col)` counts different non-NULL values. After a `LEFT JOIN`, count a column of the right table, not `*`, otherwise unmatched rows count as 1 instead of 0. → *More:* Ch 4 — Aggregate functions

Q20. Why does `SELECT dept, name, max(salary) FROM emp GROUP BY dept` fail?

After grouping, each output row stands for a whole department, and `name` has many values in it; the database does not know which one to show. Every selected column must be grouped or aggregated (PostgreSQL allows ungrouped columns only when the primary key is grouped). To get the person with the highest salary per department, use a window function or `DISTINCT ON` (see Q177 and the top-N problems). → *More:* Ch 4 — GROUP BY

Q21. How does `DISTINCT` differ from `GROUP BY`, and what is `DISTINCT ON`?

`SELECT DISTINCT a, b` and `GROUP BY a, b` without aggregates return the same rows; `GROUP BY` is needed when you also compute aggregates. PostgreSQL's `DISTINCT ON (a) … ORDER BY a, b DESC` keeps the **first row of each group** in the given order, which is the shortest way to write "latest row per customer". → *More:* Ch 2 — DISTINCT

Q22. ⋆ How do you paginate, and why is a large `OFFSET` slow?

`LIMIT 20 OFFSET 40000` still reads and throws away 40,000 rows, so deep pages get slower and rows can repeat or be skipped when data changes between requests. **Keyset pagination** remembers the last key seen and asks for the next ones, `WHERE (created_at, id) < ($1, $2) ORDER BY created_at DESC, id DESC LIMIT 20`, which an index answers at the same cost however deep the page is. → *More:* Ch 12 — OFFSET vs keyset pagination

Q23. `LIKE` vs `ILIKE` vs regular expressions? Can `LIKE '%abc'` use an index?

`LIKE` matches simple patterns (`%` any string, `_` one character) and is case-sensitive; `ILIKE` is PostgreSQL's case-insensitive version; `~` and `~*` use POSIX regular expressions. A B-tree index can serve a prefix pattern (`'abc%'`, with the `C` collation or a `text_pattern_ops` index), but a leading wildcard (`'%abc'`) cannot use it; a trigram GIN index (`pg_trgm`) can. → *More:* Ch 2 — Ranges, lists and patterns

Q24. Why does `SELECT 7 / 2` return 3 in PostgreSQL?

Both operands are integers, so it is integer division and the fraction is truncated. Cast one side, `7 / 2.0` or `7::numeric / 2`, to get 3.5. This bites ratio questions (Q134, Q232), which is why the solutions multiply by `1.0` or cast to `numeric`. → *More:* Ch 3 — Data types and conversions

Joins, subqueries and set operations

Q25. ⋆ Explain the types of joins.

INNER JOIN returns only pairs of rows that match. **LEFT JOIN** returns every left row plus matching right rows, with NULLs where none match; **RIGHT JOIN** is the mirror image. **FULL JOIN** returns all rows from both sides. **CROSS JOIN** returns every combination (a Cartesian product), and a **self join** joins a table to itself, such as employees to their managers. → *More:* Ch 5 — The four main join types at a glance

Q26. ⋆ Table A has 3 rows and B has 4 rows. How many rows can `A JOIN B` return?

An inner join returns between 0 and 12 rows (3 × 4 when every row matches every row, for example when all join keys are equal). A `LEFT JOIN` returns at least 3. A `CROSS JOIN` returns exactly 12. Duplicated join keys multiply rows, which is the "fan-out" problem behind inflated sums. → *More:* Ch 5 — Join pitfalls

Q27. ⋆ Why did my `LEFT JOIN` turn into an inner join?

A condition on the right table in `WHERE` (for example `WHERE o.status = 'paid'`) is FALSE or UNKNOWN for the NULL-padded rows, so they are filtered out. Put conditions on the right table into the `ON` clause, and keep `WHERE` for the left table. → *More:* Ch 5 — Join pitfalls

Q28. ⋆ How do you find rows in A that have no match in B (an anti-join)?

Three ways: `WHERE NOT EXISTS (SELECT 1 FROM b WHERE b.a_id = a.id)`, `LEFT JOIN b … WHERE b.a_id IS NULL`, and `NOT IN` (unsafe if `b.a_id` can be NULL). `NOT EXISTS` is the safest and PostgreSQL plans it as a hash or merge anti-join. → *More:* Ch 5 — Semi-joins and anti-joins

Q29. What is a self join? Give an example.

A table joined to itself under two aliases. Typical uses: employee and manager in the same table, comparing a day with the previous day, or finding pairs of rows (`a.id < b.id`) with something in common.

```
SELECT e.name AS employee, m.name AS manager
FROM employee e LEFT JOIN employee m ON m.id = e.manager_id;
```

→ *More:* Ch 5 — SELF JOIN

Q30. What is a correlated subquery, and how is it different from a regular one?

A regular (non-correlated) subquery can run once on its own. A **correlated** subquery refers to a column of the outer row, so it is logically evaluated once per outer row, for example "employees paid more than their department's average". PostgreSQL often rewrites these into joins, but a correlated scalar subquery in the `SELECT` list can still be slow on large tables. → *More:* Ch 6 — Correlated subqueries

Q31. ⋆ `IN` vs `EXISTS` — which is faster?

For positive tests (`IN (subquery)` vs `EXISTS`), PostgreSQL turns both into a semi-join and they usually have the same plan. The real difference is in the negative form: `NOT IN` breaks on NULLs (Q16) and is harder to optimise, so prefer `NOT EXISTS`. → *More:* Ch 6 — EXISTS and NOT EXISTS

Q32. ⋆ `UNION` vs `UNION ALL`?

`UNION` removes duplicate rows from the combined result, which costs a sort or hash. `UNION ALL` keeps every row and is cheaper. Use `UNION ALL` unless you really need de-duplication. Both need the same number of columns with compatible types. → *More:* Ch 6 — Set operations and UNION

Q33. What do `INTERSECT` and `EXCEPT` do?

`INTERSECT` returns rows present in both queries; `EXCEPT` (Oracle: `MINUS`) returns rows of the first query that are not in the second. Both remove duplicates unless you write `ALL`. Unlike `=`, they treat two NULLs as the same. → *More:* Ch 6 — EXCEPT

Q34. What is a `LATERAL` join?

A subquery in `FROM` that may refer to columns of tables listed before it, so it runs once per left row. It is the natural tool for "top 3 orders per customer" with a `LIMIT` inside, and for calling set-returning functions per row (`CROSS JOIN LATERAL unnest(…)`). → *More:* Ch 6 — LATERAL subqueries

Q35. Equi-join vs non-equi join vs natural join?

An **equi-join** matches on equality (`a.id = b.a_id`). A **non-equi join** uses another comparison, such as a sale date `BETWEEN` a price period's start and end. A **natural join** joins automatically on all same-named columns, which silently changes when someone adds a column, so avoid it and write `ON` or `USING`. → *More:* Ch 5 — USING, NATURAL JOIN and multi-table joins

Aggregation, CTEs, views and window functions

Q36. ⋆ What is a CTE, and when would you use one instead of a subquery?

A **CTE** (common table expression) is a named subquery written with `WITH name AS (…)` before the main query. It makes multi-step logic readable, lets you reference the same result more than once, and is the only way to write recursion. Since PostgreSQL 12 a CTE used once is inlined like a subquery, so it costs nothing extra; write `AS MATERIALIZED` to force it to be computed once. → *More:* Ch 7 — Common table expressions

Q37. ⋆ How does a recursive CTE work? Write one that walks an org chart.

The **anchor** query produces the starting rows; the **recursive** part joins the previous round's rows back to the table, and rounds repeat until one adds no rows. To stop cycles, use PostgreSQL 14's `CYCLE` clause (or track the visited path); `UNION` instead of `UNION ALL` helps only when a repeated row is identical, so it does not stop a cycle once you carry a changing column such as `depth`.

```
WITH RECURSIVE chain AS (
  SELECT id, name, manager_id, 1 AS depth FROM employee WHERE manager_id IS NULL
  UNION ALL
  SELECT e.id, e.name, e.manager_id, c.depth + 1
  FROM employee e JOIN chain c ON e.manager_id = c.id
)
SELECT * FROM chain ORDER BY depth, id;
```

→ *More:* Ch 7 — Hierarchies

Q38. ⋆ What is a view? What is a materialized view, and when do you use each?

A **view** is a stored query with a name; it holds no data and runs its query each time, so it is always current. A **materialized view** stores the query's result like a table, so reads are fast but the data is only as fresh as the last `REFRESH MATERIALIZED VIEW` (add `CONCURRENTLY`, which needs a unique index, to avoid blocking readers). Use views for reuse and access control, materialized views for expensive reports that may be slightly stale. → *More:* Ch 7 — Views · Materialized views

Q39. Can you `INSERT` or `UPDATE` through a view?

Yes, if the view is simple: one table, no aggregates, `DISTINCT`, `GROUP BY` or set operations. PostgreSQL makes such views updatable automatically. `WITH CHECK OPTION` refuses changes that would make the row disappear from the view, and `INSTEAD OF` triggers make complex views writable. → *More:* Ch 7 — Views

Q40. ⋆ Window function vs `GROUP BY`?

`GROUP BY` collapses each group into one output row. A **window function** (`… OVER (PARTITION BY … ORDER BY …)`) computes a value over a set of related rows but keeps every row, so you can show each employee next to the department average, a rank or a running total. → *More:* Ch 8 — What a window function is

Q41. ★ `ROW_NUMBER` vs `RANK` vs `DENSE_RANK`?

For scores 100, 90, 90, 80: `row_number()` gives 1, 2, 3, 4 (ties broken arbitrarily), `rank()` gives 1, 2, 2, 4 (gaps after ties) and `dense_rank()` gives 1, 2, 2, 3 (no gaps). Use `dense_rank` for "Nth highest distinct value", `rank` for competition placing, and `row_number` for de-duplication or exactly one row per group. → *More:* Ch 8 — Ranking functions

Q42. ★ What do `LAG` and `LEAD` do?

`lag(col, n, default)` returns the value from n rows **before** the current row in the window's order, and `lead` from n rows **after**. They replace self joins for "compare with the previous day", "time since last login" and "month-over-month growth". → *More:* Ch 8 — Value (offset) functions

Q43. What is a window frame? Why does my running total repeat on tied dates, and why does `LAST_VALUE` return the current row?

The **frame** is the subset of the partition that an aggregate window function sees. With `ORDER BY` and no frame clause, the default is `RANGE BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW`, which includes all rows tied with the current one (so ties get the same running total) and stops at the current row (so `last_value` sees only up to here). Write `ROWS BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW` or `… AND UNBOUNDED FOLLOWING` when you mean that. → *More:* Ch 8 — Frames

Q44. Why can't I put a window function in `WHERE`?

Window functions are computed after `WHERE`, `GROUP BY` and `HAVING` (Q13), so they do not exist yet when rows are filtered. Compute them in a subquery or CTE, then filter outside: `SELECT * FROM (SELECT …, rank() OVER (…) AS r FROM t) x WHERE r <= 3`. → *More:* Ch 8 — Top-N per group

Q45. ★ What is the "gaps and islands" problem, and the trick that solves it?

Islands are runs of consecutive values (days, ids) and **gaps** are the holes between them. Within a run, `value − row_number()` (ordered by value) is constant, so grouping by that difference puts each run in its own group; then `min`, `max` and `count` describe each island. Used for login streaks, consecutive numbers and contiguous date ranges (Q153, Q199, Q200). → *More:* Ch 8 — Gaps and islands

Q46. ★ How do you remove duplicate rows from a table, keeping one?

Number the copies with `row_number() OVER (PARTITION BY the_duplicate_columns ORDER BY id)` and delete those numbered above 1, or use a self join: `DELETE FROM t a USING t b WHERE a.email = b.email AND a.id > b.id`. Then add a unique constraint so it cannot happen again. → *More:* Ch 8 — Top-N per group and de-duplication

Q47. What are `GROUPING SETS`, `ROLLUP` and `CUBE`?

They compute several groupings in one query. `ROLLUP (year, month)` gives per-month rows, per-year subtotals and a grand total; `CUBE (a, b)` gives every combination of subtotals; `GROUPING SETS` lists exactly the groupings you want. `grouping(col)` tells you which rows are subtotals. → *More:* Ch 4 — GROUPING SETS, ROLLUP and CUBE

Q48. ★ How do you pivot rows into columns in PostgreSQL?

Use conditional aggregation: `sum(amount) FILTER (WHERE month = 'Jan') AS jan` (or `sum(CASE WHEN … THEN amount END)` in other databases), one expression per output column. The `crosstab` function from the `tablefunc` extension also works. A dynamic set of columns needs dynamic SQL, because a query's columns must be known before it runs. → *More:* Ch 14 — Pivot and unpivot

Normalization and data modeling

Q49. ★ What is normalization, and why do we normalize?

Normalization organises columns into tables so that each fact is stored in exactly one place. It removes redundancy and the **anomalies** that come with it: an *update anomaly* (a change made in one copy but not another), an *insertion anomaly* (you cannot record a fact without an unrelated one) and a *deletion anomaly* (deleting one fact loses another). → *More:* Ch 11 — Anomalies

Q50. ★ Explain 1NF, 2NF, 3NF and BCNF with an example.

1NF: every column holds one atomic value (no lists like `'red,blue'`, no repeating groups). **2NF**: 1NF, and no non-key column depends on only part of a composite key (in `order_line(order_id, product_id, product_name)`, `product_name` depends only on `product_id`, so it moves to `product`). **3NF**: 2NF, and no non-

key column depends on another non-key column (`zip → city`). **BCNF**: for every functional dependency X → Y, X is a superkey. → *More:* Ch 11 — 1NF · 2NF · 3NF · BCNF

Q51. What is a functional dependency?

`X → Y` means that rows with the same X always have the same Y; for example `employee_id → department_id` . Keys and normal forms are all defined in terms of functional dependencies. → *More:* Ch 11 — Functional dependencies

Q52. What are 4NF and 5NF in one sentence each?

4NF removes independent multi-valued facts stored in the same table (a teacher's subjects and languages in one table make every combination appear). **5NF** splits a table that can be rebuilt without loss only from three or more smaller tables (a join dependency). They are rarely needed in practice but come up in theory rounds. → *More:* Ch 11 — 4NF · 5NF

Q53. ⋆ What is denormalization, and when would you do it?

Deliberately storing redundant data (a cached `order_total` , a copied customer name, a summary table) to make reads cheaper. Do it only for a measured read bottleneck, and decide how the copies stay correct: the same transaction, a trigger, or a materialized view. Analytics schemas (star schemas) are denormalized by design. → *More:* Ch 11 — Denormalization

Q54. ⋆ How do you model a many-to-many relationship?

With a **junction** (association) table holding two foreign keys, for example `enrolment(student_id, course_id)` , with the pair as the primary key (or a surrogate PK plus `UNIQUE` on the pair). Attributes of the relationship itself, such as `enrolled_on` or `grade` , live in the junction table. → *More:* Ch 10 — Relationships

Q55. What is an ER diagram? What are cardinality and participation?

An **ER (entity–relationship) diagram** draws the entities (things you store), their attributes and the relationships between them. **Cardinality** says how many instances can be related (1:1, 1:N, M:N); **participation** says whether every instance must take part (total) or not (partial), which becomes `NOT NULL` on the foreign key. → *More:* Ch 10 — The ER model and its notations

Q56. Star schema vs snowflake schema? OLTP vs OLAP?

OLTP (online transaction processing) systems run many small reads and writes on normalized tables. **OLAP** (online analytical processing) systems run big scans and aggregations, usually on a **star schema**: a central **fact** table (one row per sale) with foreign keys to denormalized **dimension** tables (date, product, store). A **snowflake schema** normalizes the dimensions into sub-tables, which saves space but costs more joins. → *More:* Ch 11 — Denormalization · Ch 16 — Big data and distributed processing

Q57. How would you store a tree (categories, comments, org chart) in SQL?

The simplest is an **adjacency list**: a `parent_id` column, queried with a recursive CTE (Q37). Alternatives trade write cost for read speed: a **materialized path** (`'1/4/9'` , or PostgreSQL's `ltree`), **nested sets** (left/right numbers, fast subtree reads, slow inserts) and a **closure table** (one row per ancestor–descendant pair). → *More:* Ch 7 — Hierarchies

Q58. Design the schema for a simple e-commerce site (or a library, or Instagram).

Start from the questions the system must answer, list entities (customer, product, order, order_line, payment), pick keys, draw relationships (an order has many lines; a line references one product), normalize to 3NF, then add constraints and indexes for the main queries. Keep the price paid on `order_line` (a historical fact), not only on `product` . Be ready to explain one denormalization and one index choice. → *More:* Ch 17 — Schema design case studies

Indexes, EXPLAIN and query optimization

Q59. ⋆ What is an index, and how does it speed up queries?

An index is a separate, sorted data structure that maps column values to row locations, like a book's index. Instead of reading every row (a sequential scan), the database walks the index to the few matching rows. The price is extra storage and slower `INSERT` / `UPDATE` / `DELETE` , because every index must be updated too. → *More:* Ch 12 — What an index is

Q60. ⋆ How does a B-tree (B+ tree) index work?

It is a balanced tree whose pages hold many sorted keys, so even millions of rows need only 3–4 page reads from root to leaf. Leaf pages are linked in order, which makes range queries (`BETWEEN` , `>`), `ORDER BY` and prefix `LIKE` efficient. It is PostgreSQL's default index type. → *More:* Ch 12 — B-trees and B+ trees

Q61. ⋆ Clustered vs non-clustered index?

A **clustered** index stores the table's rows themselves in index order, so there can be only one (InnoDB's primary key, SQL Server's clustered index). A **non-clustered** (secondary) index is separate and points to the rows. PostgreSQL tables are heaps, so every index is non-clustered; the `CLUSTER` command reorders a table once but the order is not maintained afterwards. → *More:* Ch 9 — Indexes (in brief)

Q62. What index types does PostgreSQL have, and when do you use each?

B-tree: equality, ranges and sorting (the default). **Hash**: equality only. **GIN**: values that contain many items, such as JSONB, arrays and full-text search. **GiST** and **SP-GiST**: geometric data, ranges, nearest-neighbour searches. **BRIN**: very large tables whose values follow the physical order, such as an append-only log by time; tiny but coarse. → *More:* Ch 12 — GIN, GiST, BRIN

Q63. ⋆ What is a composite index, and does column order matter?

An index on several columns, sorted by the first column, then the second within it, and so on. Order matters: an index on `(a, b)` helps `WHERE a = ?` and `WHERE a = ? AND b > ?` , but not `WHERE b = ?` alone (the **leftmost-prefix** rule). Put equality columns first, then the range or sort column. PostgreSQL 18 can sometimes "skip scan" a missing leading column, but do not rely on it. → *More:* Ch 12 — Composite indexes and column order

Q64. What are covering, partial and expression indexes?

A **covering** index holds every column a query needs (extra columns can be added with `INCLUDE`), so PostgreSQL can answer with an **index-only scan** without visiting the table. A **partial** index indexes only rows matching a `WHERE` (`WHERE status = 'pending'`), so it stays small. An **expression** index indexes a computed value such as `lower(email)` , so queries filtering on that expression can use it. → *More:* Ch 12 — Covering, partial and expression indexes

Q65. ⋆ Why is my query not using the index?

Common causes: a function or cast on the column (`WHERE date(created_at) = …` , `WHERE lower(email) = …` without an expression index), a leading wildcard `LIKE '%x'` , a type mismatch, a filter on a non-leading column of a composite index, `OR` across different columns, or a filter that matches so many rows that a sequential scan is cheaper. Also check that statistics are current (`ANALYZE`). → *More:* Ch 12 — Anti-patterns that disable indexes

Q66. Should you index every column? What is selectivity?

No. Each index slows writes, takes space and memory, and may never be used. **Selectivity** is the fraction of rows a condition keeps; an index helps when it keeps few rows (an e-mail lookup) and not when it keeps many (`is_active = true` on 95% of rows). Index the columns used in frequent `WHERE` , `JOIN` and `ORDER BY` clauses, and foreign keys that are joined or whose parents are deleted. → *More:* Ch 12 — What an index is

Q67. ⋆ How do you read `EXPLAIN` / `EXPLAIN ANALYZE` ?

`EXPLAIN` shows the plan the optimizer chose as a tree of nodes with estimated costs and row counts; `EXPLAIN (ANALYZE, BUFFERS)` also runs the query and adds real times, real row counts, loops and pages read. Read from the innermost node outwards, look for the nodes with the largest actual time, and compare estimated with actual rows: a big difference means bad statistics and usually a bad join choice. → *More:* Ch 12 — Reading EXPLAIN

Q68. Seq scan vs index scan vs index-only scan vs bitmap scan?

A **sequential scan** reads the whole table. An **index scan** walks the index and fetches each matching row from the table. An **index-only scan** answers from the index alone (needs a covering index and an up-to-date visibility map). A **bitmap scan** first collects matching row locations from one or more indexes, sorts them by page, then reads each page once; it suits medium selectivity and `OR` / `AND` of indexes. → *More:* Ch 12 — Reading EXPLAIN

Q69. Nested loop vs hash join vs merge join?

A **nested loop** looks up each outer row in the inner input, which is best when the outer side is small and the inner side is indexed. A **hash join** builds a hash table on the smaller input and probes it with the other, which is best for large equality joins. A **merge join** walks two inputs already sorted on the key, which suits large inputs that are already sorted (or cheap to sort); like a hash join, it handles only equality conditions. → *More:* Ch 12 — Join algorithms

Q70. ⋆ A query that used to be fast is now slow. How do you troubleshoot it?

1. Get the real plan with `EXPLAIN (ANALYZE, BUFFERS)` and compare with the old one. 2) Check estimates vs actual rows; run `ANALYZE` if statistics are stale. 3) Look for sequential scans on big tables, sorts or hashes spilling to disk (raise `work_mem` or add an index), and nested loops with huge loop counts. 4) Check for lock waits (`pg_stat_activity` , `pg_locks`), table bloat and data growth. 5) Fix the query or index, then measure again; `pg_stat_statements` finds the worst queries overall. → *More:* Ch 12 — A tuning workflow

Q71. ⋆ What is the N+1 query problem?

Code runs one query to load a list, then one more query per item (for each of 100 orders, fetch its customer), which makes 101 round trips. Fix it with a join, one batched query (`WHERE id = ANY($1)`), or the ORM's eager loading. → *More:* Ch 14 — ORMs vs raw SQL

Q72. How does the optimizer estimate rows? What do `ANALYZE` and `VACUUM` do?

The planner uses per-column statistics (row counts, NULL fraction, distinct values, most common values, histograms) gathered by `ANALYZE` . `VACUUM` removes dead row versions left by MVCC (Q85) so space can be reused, and updates the visibility map that index-only scans need. **Autovacuum** runs both automatically; tables with very high churn may need tuned thresholds. → *More:* Ch 12 — Statistics

Q73. Give five ways to make a slow SQL query faster.

Add the right index (composite, covering or partial); remove functions from indexed columns in `WHERE` ; select only needed columns instead of `SELECT *` ; replace `OFFSET` paging with keyset paging; pre-aggregate with a materialized view or summary table; batch N+1 queries into one; and for huge tables, partition so queries touch fewer rows. Always confirm with `EXPLAIN ANALYZE` .

Transactions, isolation and locking

Q74. ⋆ What is a transaction? Explain ACID.

A **transaction** is a group of statements that succeed or fail as one unit. **Atomicity**: all of its changes happen or none do. **Consistency**: it moves the database from one valid state to another, respecting constraints. **Isolation**: concurrent transactions do not see each other's half-done work (to the degree the isolation level promises). **Durability**: once committed, changes survive a crash, because they were first written to the write-ahead log. → *More:* Ch 13 — What a transaction is

Q75. ⋆ What are dirty reads, non-repeatable reads and phantom reads?

A **dirty read** sees another transaction's uncommitted change. A **non-repeatable read** reads the same row twice and gets different values because another transaction committed an update in between. A **phantom read** runs the same `WHERE` query twice and gets a different *set* of rows because another transaction inserted or deleted matching rows. A **lost update** and **write skew** are two more anomalies (Q78, Q79). → *More:* Ch 13 — Isolation

Q76. ⋆ Name the four isolation levels and what each prevents.

Level	Dirty read	Non-repeatable read	Phantom
Read Uncommitted	possible (not in PostgreSQL)	possible	possible
Read Committed	prevented	possible	possible
Repeatable Read	prevented	prevented	possible in the standard, prevented in PostgreSQL
Serializable	prevented	prevented	prevented

Serializable also prevents write skew: the result equals some one-at-a-time order of the transactions. → *More:* Ch 13 — Isolation

Q77. ⋆ What is PostgreSQL's default isolation level, and how does PostgreSQL implement the levels?

Read Committed is the default (MySQL InnoDB defaults to Repeatable Read). Each statement sees a snapshot taken when the statement starts. Repeatable Read takes one snapshot for the whole transaction (snapshot isolation), so phantoms cannot appear and concurrent updates of the same row fail with a serialization error. Serializable adds **SSI** (serializable snapshot isolation), which tracks read/write dependencies and aborts one transaction with SQLSTATE `40001` when an anomaly is possible; the application must retry. Read Uncommitted behaves like Read Committed. → *More:* Ch 13 — Watching isolation anomalies

Q78. ⋆ What is a lost update, and how do you prevent it?

Two transactions read the same value, both compute a new value from it and both write, so the first write is lost (two clerks both read stock 10 and both write 9). Prevent it with an atomic update (`UPDATE … SET stock = stock - 1 WHERE stock > 0`), a row lock (`SELECT … FOR UPDATE`), optimistic locking with a version column (`UPDATE … WHERE id = ? AND version = ?`, then check the row count), or Repeatable Read/Serializable with retry. → *More:* Ch 13 — Lost updates

Q79. What is write skew? Give an example.

Two transactions read the same data, each checks a rule, then each updates a *different* row, and together they break the rule. Example: the rule is "at least one doctor on call"; two doctors on call both see two on call and both go off call. Row locks on the rows you write do not stop it; use Serializable isolation, or lock the rows you read with `SELECT … FOR UPDATE`. → *More:* Ch 13 — Write skew and Serializable isolation

Q80. ⋆ Optimistic vs pessimistic locking?

Pessimistic locking takes locks before working (`SELECT … FOR UPDATE`), so others wait; it is safe under high contention but reduces concurrency and can deadlock. **Optimistic** locking works without locks and checks at write time whether someone else changed the row (a `version` column or timestamp), retrying on conflict; it is best when conflicts are rare, such as web forms. → *More:* Ch 13 — Lost updates

Q81. Shared vs exclusive locks? Row-level vs table-level locks?

A **shared** lock lets others read but not modify; an **exclusive** lock blocks other writers (in PostgreSQL readers are never blocked by row locks, thanks to MVCC). **Row-level** locks (`FOR UPDATE`, `FOR SHARE`, and those taken by `UPDATE` / `DELETE`) protect single rows; **table-level** locks (eight modes, from `ACCESS SHARE` taken by `SELECT` up to `ACCESS EXCLUSIVE` taken by most `ALTER TABLE` and `DROP`) protect the table's structure. → *More:* Ch 13 — Lock-based concurrency control

Q82. ⋆ What is a deadlock, and how do you avoid one?

Two transactions each hold a lock the other needs, so both wait forever. PostgreSQL checks for cycles after `deadlock_timeout` (1 s by default) and aborts one transaction with error `40P01`. Avoid them by locking rows in a consistent order (for example by primary key), keeping transactions short, and retrying the aborted one. → *More:* Ch 13 — Deadlocks

Q83. What is two-phase locking (2PL)?

A locking protocol in which a transaction first only acquires locks (growing phase) and then only releases them (shrinking phase); strict 2PL holds write locks until commit. It guarantees serializable schedules, at the cost of blocking and deadlocks. MySQL's and SQL Server's Serializable levels use locking like this, while PostgreSQL uses SSI. → *More:* Ch 13 — Lock-based concurrency control

Q84. ⋆ How do you build a job queue in SQL so workers don't take the same job?

Each worker runs, inside a transaction, `SELECT id FROM jobs WHERE status = 'ready' ORDER BY id LIMIT 1 FOR UPDATE SKIP LOCKED`, processes the job, marks it done and commits. `SKIP LOCKED` makes each worker skip rows already locked by others instead of waiting, so workers never block each other. `NOWAIT` instead raises an error at once. → *More:* Ch 13 — Row locks for work queues

Q85. ⋆ What is MVCC?

Multi-version concurrency control: an `UPDATE` writes a new version of the row instead of overwriting it, and each transaction reads the versions visible in its snapshot. So readers never block writers and

writers never block readers. The cost is dead versions that `VACUUM` must clean up, and table bloat if it falls behind (for example because of a long-running transaction). → *More:* Ch 13 — MVCC

Q86. How is durability guaranteed? What is the WAL?

Before a change is applied to the data files, it is written to the **write-ahead log** (WAL) and the WAL is flushed to disk at commit. After a crash, the database replays the WAL from the last checkpoint to redo committed work. The same WAL stream is what replicas receive (Q101) and what point-in-time recovery replays. → *More:* Ch 13 — Durability

Q87. What are savepoints?

Named points inside a transaction: `SAVEPOINT s1; … ROLLBACK TO SAVEPOINT s1;` undoes only the work after `s1` and keeps the transaction open. In PostgreSQL an error aborts the whole transaction unless you roll back to a savepoint, which is how drivers implement "continue after an error". → *More:* Ch 13 — Atomicity

Q88. A bank transfer: write it so money is never lost or created.

Do both updates in one transaction, update rows in a fixed order (lower account id first) to avoid deadlocks, and let a `CHECK (balance >= 0)` constraint reject overdrafts.

```
BEGIN;
UPDATE account SET balance = balance - 100 WHERE id = 1;
UPDATE account SET balance = balance + 100 WHERE id = 2;
COMMIT;   -- any error before this: ROLLBACK, nothing changed
```

→ *More:* Ch 13 — Putting it together

Stored procedures, triggers, security and SQL injection

Q89. ⋆ Function vs stored procedure in PostgreSQL?

A **function** (`CREATE FUNCTION`) returns a value or a set of rows and is called inside a query (`SELECT f(x)`); it runs inside the caller's transaction and cannot commit. A **procedure** (`CREATE PROCEDURE` , PostgreSQL 11+) is invoked with `CALL` , returns no value (only `OUT` parameters, 14+) and can `COMMIT` or `ROLLBACK` inside its body, which suits batch jobs. Both can be written in SQL or PL/pgSQL. → *More:* Ch 14 — Stored procedures

Q90. What are the pros and cons of stored procedures?

Pros: fewer network round trips, logic close to the data, one place for rules shared by several applications, and permissions granted on the procedure rather than the tables. Cons: harder to version, test, debug and deploy than application code, tied to one database's language, and they can hide business logic where developers do not look. → *More:* Ch 14 — Stored procedures

Q91. ⋆ What is a trigger? `BEFORE` vs `AFTER` vs `INSTEAD OF` , row vs statement level?

A trigger runs a function automatically when a table is modified. `BEFORE` triggers can change or cancel the row (fill `updated_at` , normalise data); `AFTER` triggers see the final row and suit audit logs; `INSTEAD OF` triggers replace the operation on a view. A **row-level** trigger runs once per affected row; a **statement-level** trigger runs once per statement, however many rows. → *More:* Ch 14 — Triggers

Q92. When should you avoid triggers?

When the logic is business logic that developers need to see and test; triggers are invisible from the application, can cascade into other triggers, slow down bulk writes and make debugging hard. They are a good fit for audit trails, `updated_at` columns and enforcing invariants that constraints cannot express. → *More:* Ch 14 — Triggers

Q93. ⋆ What is SQL injection, and how do you prevent it?

An attacker puts SQL syntax into input that the application concatenates into a query, for example a name of `' OR '1'='1` that turns `WHERE name = '…'` into a condition that is always true, or one that adds `; DROP TABLE …` . Prevent it with **parameterized queries** (prepared statements with placeholders), where the value is sent separately and never parsed as SQL. Also allow-list identifiers such as sort columns (they cannot be parameters), give the application role only the privileges it needs, and never build dynamic SQL from raw input (in PL/pgSQL use `format('%I', …)` and `USING`). → *More:* Ch 14 — SQL injection and parameterized queries

Q94. Does an ORM protect you from SQL injection?

Mostly, because ORMs generate parameterized queries. You are exposed again as soon as you use raw SQL or string-built fragments (`order_by(user_input)`, string-formatted native queries). Escaping by hand is not a substitute for parameters. → *More:* Ch 14 — ORMs vs raw SQL

Q95. What are `GRANT`, roles and row-level security?

A **role** is a user or group; `GRANT SELECT ON orders TO analyst` gives it a privilege and `REVOKE` takes it away. Grant the least privilege needed. **Row-level security** (RLS) adds policies that filter which rows a role can see or change, for example `USING (tenant_id = current_setting('app.tenant')::int)` for multi-tenant apps. → *More:* Ch 14 — Security

Q96. What is connection pooling, and why does PostgreSQL need it?

Each PostgreSQL connection is a separate server process using memory, so thousands of short-lived connections waste resources. A **pool** (in the application, or PgBouncer in front of the database) keeps a small set of open connections and lends them out per request or per transaction. → *More:* Ch 14 — Connection pooling

Q97. JSON vs JSONB in PostgreSQL? When would you store JSON in a relational database?

`json` stores the text as given; `jsonb` stores a parsed binary form that drops whitespace and duplicate keys, is faster to query and can be indexed with GIN. Use JSONB for truly variable attributes (product specs, event payloads), but keep fields you filter, join or constrain on as real columns. → *More:* Ch 14 — JSON and JSONB

Partitioning, replication, sharding and SQL vs NoSQL

Q98. ⋆ What is partitioning? Range vs list vs hash partitioning?

Partitioning splits one large table into smaller physical tables (partitions) that queries still see as one. **Range** partitioning splits by intervals (one partition per month), **list** by explicit values (per country), and **hash** by a hash of the key (even spread). Benefits: **partition pruning** (queries touch only relevant partitions), and dropping old data with `DROP` / `DETACH PARTITION` instead of a huge `DELETE`. PostgreSQL has declarative partitioning since version 10. → *More:* Ch 15 — Partitioning

Q99. ⋆ Partitioning vs sharding vs replication?

Partitioning splits a table into pieces on the same server. **Sharding** puts those pieces on different servers, so writes and storage scale out. **Replication** keeps full copies of the same data on several servers for read scaling and high availability. They are combined in practice: each shard is usually replicated. → *More:* Ch 15 — Sharding

Q100. ⋆ How do you choose a shard key? What problems does sharding bring?

Pick a key that spreads load evenly and keeps most queries on one shard, often the tenant or user id. Avoid monotonically increasing keys with range sharding (all new writes hit one shard) and low-cardinality keys (hot spots). Costs: cross-shard queries and joins, distributed transactions, resharding when shards fill up (consistent hashing helps), and global uniqueness. Citus is the usual way to shard PostgreSQL. → *More:* Ch 15 — Sharding

Q101. ⋆ Synchronous vs asynchronous replication? What is replication lag?

Synchronous replication makes the primary wait until a replica confirms before reporting commit, so no committed data is lost on failover but every write is slower. **Asynchronous** replication commits first and ships changes afterwards: faster, but recent commits can be lost if the primary dies. **Replication lag** is how far a replica is behind; reading from a lagging replica can show stale data, so read-your-own-writes queries should go to the primary. → *More:* Ch 15 — Replication

Q102. Physical (streaming) vs logical replication in PostgreSQL?

Physical streaming replication ships WAL bytes, making an exact copy of the whole cluster; the replica is read-only and must run the same major version. **Logical replication** (publications and subscriptions, PostgreSQL 10+) sends row changes for chosen tables, works across major versions and lets the subscriber have its own writes and indexes; it suits upgrades and feeding other systems. → *More:* Ch 15 — Replication

Q103. Single-leader vs multi-leader vs leaderless replication?

Single-leader: one primary takes all writes and followers copy it (PostgreSQL, MySQL); simple, no write conflicts. **Multi-leader**: several nodes accept writes (multi-region setups) and must resolve conflicts. **Leaderless**: clients write to several replicas and read with quorums (`W + R > N`), as in Cassandra and DynamoDB. → *More:* Ch 15 — Replication

Q104. ⋆ Explain the CAP theorem (and PACELC).

When a network **partition** separates nodes, a distributed store must choose between **consistency** (every read sees the latest write, so some requests are refused) and **availability** (every request gets an answer, possibly stale). Without a partition there is no forced choice. **PACELC** adds: Else (no partition) the trade-off is **latency** vs **consistency**. → *More:* Ch 15 — The CAP theorem

Q105. Strong vs eventual consistency?

Strong (linearizable) consistency: once a write is acknowledged, every later read sees it. **Eventual** consistency: replicas converge if writes stop, but a read may return old data meanwhile. In between are read-your-writes, monotonic reads and causal consistency. → *More:* Ch 15 — Consistency models

Q106. ⋆ SQL vs NoSQL — how do you choose?

Choose relational (SQL) for structured data with relationships, ad-hoc queries, joins, constraints and multi-row transactions (orders, payments, inventory). Choose NoSQL for a specific access pattern at very large scale or with a flexible shape: key-value for caches and sessions, document stores for self-contained aggregates, wide-column for huge write-heavy time-ordered data, graph for many-hop relationships. Many systems use both; PostgreSQL with JSONB covers many "we need a document store" cases. → *More:* Ch 1 — Relational vs non-relational · Ch 16 — Choosing a database

Q107. ACID vs BASE?

ACID (Q74) is the transactional guarantee of relational databases. **BASE** (Basically Available, Soft state, Eventually consistent) describes many distributed NoSQL stores that stay available and accept temporary inconsistency. The line is blurred today: MongoDB has multi-document transactions, and NewSQL databases (CockroachDB, Spanner, YugabyteDB) give ACID across nodes. → *More:* Ch 1 — Relational vs non-relational

Q108. What is two-phase commit (2PC)?

A protocol that commits one transaction across several databases: the coordinator asks every participant to **prepare** (make the change durable and promise to commit), then sends **commit** only if all said yes, otherwise **abort**. It is blocking: if the coordinator fails after prepare, participants hold locks until it returns. Microservices usually prefer sagas with compensating actions. → *More:* Ch 15 — Consensus and leader election

Q109. How do you achieve high availability for PostgreSQL?

A primary with one or more streaming replicas, an automatic failover manager (Patroni, or a managed service), a connection endpoint that follows the primary, synchronous replication if no data loss is acceptable, and tested backups with WAL archiving for point-in-time recovery. A replica is not a backup: it copies mistakes such as a `DROP TABLE` within seconds. → *More:* Ch 15 — High availability

Q110. What are read replicas, and what problems do they bring?

Copies that serve read-only queries so the primary handles fewer reads. Problems: stale reads because of lag, routing logic in the application, and queries on a replica can be cancelled when they conflict with changes being replayed. They do not help with write load; sharding does. → *More:* Ch 15 — Replication

Scenario and senior-level questions

Q111. ⋆ How would you scale a database that is getting slow as traffic grows?

In order: measure (top queries from `pg_stat_statements`); fix queries and indexes; add caching (Redis) for hot reads; add read replicas; partition the biggest tables; move analytics to a separate store; scale the server vertically; and only then shard. Each step adds complexity, so stop when the bottleneck is gone. → *More:* Ch 15 — Distributed databases: the big picture

Q112. How do you add a column or index to a huge table without downtime?

`ALTER TABLE … ADD COLUMN` with no default, or with a constant default (PostgreSQL 11+), is instant. Create indexes with `CREATE INDEX CONCURRENTLY` (no write lock, cannot run inside a transaction). Add

constraints as `NOT VALID` first, then `VALIDATE CONSTRAINT` separately. Set a short `lock_timeout` so a blocked `ALTER` does not queue every other query behind it, and backfill data in small batches. → *More:* Ch 9 — ALTER

Q113. How do you delete 100 million old rows safely?

If the table is partitioned by time, drop or detach old partitions, which is instant. Otherwise delete in batches (for example 10,000 rows per transaction by key range), pausing between batches so replicas and autovacuum keep up, then let `VACUUM` reclaim the space. One huge `DELETE` holds locks for a long time, bloats the table and floods the WAL. → *More:* Ch 15 — Partitioning

Q114. UUID vs auto-increment primary key?

`bigint` identity keys are small, fast to index and ordered, but reveal counts and need a central generator. UUIDs can be generated anywhere (useful for distributed systems and offline clients) but are 16 bytes, and random UUIDv4 values scatter inserts across the index. Time-ordered UUIDv7 (`uuidv7()` built in from PostgreSQL 18) keeps inserts local. → *More:* Ch 9 — Generated keys

Q115. How do you store and query time zones correctly?

Store instants as `timestamptz` (PostgreSQL stores UTC and converts on display using the session's `TimeZone`), and convert with `AT TIME ZONE 'Asia/Kolkata'` when you need local dates, for example "orders per local day". Use plain `timestamp` only for wall-clock values that are not instants, such as "store opens at 09:00". → *More:* Ch 3 — Date and time functions

Q116. How do you prevent double booking (two people booking the same seat or room)?

Make the database enforce it: a `UNIQUE (show_id, seat_no)` constraint for seats, or an exclusion constraint for time ranges, `EXCLUDE USING gist (room_id WITH =, during WITH &&)` (needs the `btree_gist` extension). A concurrent second booking then fails with a constraint error, whatever the application did. `SELECT … FOR UPDATE` on the seat row is the locking alternative. → *More:* Ch 12 — GIN, GiST, BRIN · Ch 13 — Write skew

Q117. How would you implement soft deletes, and what are the pitfalls?

Add `deleted_at timestamptz` and filter `WHERE deleted_at IS NULL` everywhere (a view helps). Pitfalls: every query must remember the filter, unique constraints must become partial (`UNIQUE (email) WHERE deleted_at IS NULL`), foreign keys still point at "deleted" rows, and the table keeps growing. Archiving deleted rows to another table is the alternative. → *More:* Ch 9 — DELETE · Ch 12 — Covering, partial and expression indexes

Q118. How do you keep a search index or cache in sync with the database?

Write to the database only, and publish changes from it: either the **transactional outbox** (insert an event row in the same transaction, a relay publishes it) or **change data capture** (CDC) that reads the WAL through logical decoding (Debezium). Dual writes from the application (database, then cache) drift when one write fails. → *More:* Ch 16 — Elasticsearch and Redis

Q119. What happens, step by step, when you run a `SELECT`?

The client sends the text; the **parser** checks syntax and builds a parse tree; the **analyzer/rewriter** resolves names and expands views; the **planner/optimizer** estimates the cost of alternative plans using statistics and picks the cheapest; the **executor** runs the plan, reading pages through the **buffer pool** (shared buffers) or from disk, and streams rows back. → *More:* Ch 1 — Inside a DBMS

Q120. Which PostgreSQL features after version 14 are worth knowing?

15: `MERGE`, `UNIQUE NULLS NOT DISTINCT`, `security_invoker` views. 16: `ANY_VALUE()`, logical replication from a standby, more parallelism. 17: `JSON_TABLE` and SQL/JSON functions, `MERGE … RETURNING`, incremental backups. 18: asynchronous I/O, B-tree skip scan, `uuidv7()`, virtual generated columns (the default), `OLD` / `NEW` in `RETURNING`, temporal `WITHOUT OVERLAPS` keys. → *More:* Ch 20 — Dialect differences

Part B — Coding problems

How to answer any of these in an interview: restate the output (one row per *what*?), name the pattern, write the query in small steps (a CTE per step), then check the edge cases: NULLs, ties, empty groups that must show 0, duplicates and date boundaries. The method is in §18.1 and the pattern catalogue in §19.11.

SQL 50 — Select

Q121. ⋆ Recyclable and Low Fat Products · LeetCode 1757 · Easy

Filter on both flag columns with `AND`: `WHERE low_fats = 'Y' AND recyclable = 'Y'`. → *Solved in the book:* Ch 18 — P3 · Green products

Q122. ⋆ Find Customer Referee · LeetCode 584 · Easy

`referee_id <> 2` is UNKNOWN for customers with no referee, so they vanish; add `OR referee_id IS NULL` (or write `referee_id IS DISTINCT FROM 2`). → *Solved in the book:* Ch 18 — P4 · Everyone outside department 3

Q123. Big Countries · LeetCode 595 · Easy

One `OR` filter: `WHERE area >= 3000000 OR population >= 25000000`, returning `name, population, area`. → *Solved in the book:* Ch 18 — P1 · Big cities

Q124. Article Views I · LeetCode 1148 · Easy

Keep rows where `author_id = viewer_id`, then `SELECT DISTINCT author_id AS id … ORDER BY id`, because one author can view their own articles many times. → *Solved in the book:* Ch 18 — P5 · Self-assigned tasks

Q125. Invalid Tweets · LeetCode 1683 · Easy

`WHERE char_length(content) > 15`. `length` / `char_length` count characters; `octet_length` would count bytes. → *Solved in the book:* Ch 18 — P6 · Long comments

SQL 50 — Basic joins

Q126. Replace Employee ID With The Unique Identifier · LeetCode 1378 · Easy

`LEFT JOIN`, because employees without a unique id must still appear (with NULL).

```sql
SELECT u.unique_id, e.name
FROM Employees e
LEFT JOIN EmployeeUNI u ON u.id = e.id;
```

Q127. Product Sales Analysis I · LeetCode 1068 · Easy

Plain inner join of `Sales` to `Product` on `product_id`, selecting `product_name, year, price`. → *Solved in the book:* Ch 18 — P17 · Product name for every sale

Q128. ⋆ Customer Who Visited but Did Not Make Any Transactions · LeetCode 1581 · Easy

Anti-join visits to transactions, then count the remaining visits per customer.

```sql
SELECT v.customer_id, count(*) AS count_no_trans
FROM Visits v
WHERE NOT EXISTS (SELECT 1 FROM Transactions t WHERE t.visit_id = v.visit_id)
GROUP BY v.customer_id;
```

Q129. ⋆ Rising Temperature · LeetCode 197 · Easy

Self-join each day to the calendar day before (`date - 1`), not to the previous row, because days can be missing. (`lag()` would compare with the previous *row*, which is wrong when a day is missing.)

```sql
SELECT w.id
FROM Weather w
JOIN Weather y ON y.recordDate = w.recordDate - 1
WHERE w.temperature > y.temperature;
```

Q130. Average Time of Process per Machine · LeetCode 1661 · Easy

Pair each process's `start` and `end` rows (self-join or conditional aggregation), subtract, then average per machine; cast to `numeric` before `round(…, 3)` because `timestamp` is a float column. → *Solved in the book:* Ch 18 — P18 · Average process time per machine

Q131. ⋆ Employee Bonus · LeetCode 577 · Easy

`LEFT JOIN` the bonus and keep `bonus < 1000 OR bonus IS NULL`; forgetting the NULL case drops employees with no bonus.

```sql
SELECT e.name, b.bonus
FROM Employee e
LEFT JOIN Bonus b ON b.empId = e.empId
WHERE b.bonus < 1000 OR b.bonus IS NULL;
```

Q132. ⋆ Students and Examinations · LeetCode 1280 · Easy

`CROSS JOIN` students × subjects to build every pair, `LEFT JOIN` the exams, and `count(e.subject_name)` so missing exams count 0. → *Solved in the book:* Ch 18 — P20 · Exam sittings per student and subject

Q133. ⋆ Managers with at Least 5 Direct Reports · LeetCode 570 · Medium

Self-join manager to employees, group by the manager, keep groups of 5 or more.

```sql
SELECT m.name
FROM Employee m
JOIN Employee e ON e.managerId = m.id
GROUP BY m.id, m.name
HAVING count(*) >= 5;
```

Q134. ⋆ Confirmation Rate · LeetCode 1934 · Medium

`LEFT JOIN` confirmations to every signup and average a 0/1 flag; a user with no messages gets NULL, which `COALESCE` turns into 0.

```sql
SELECT s.user_id,
       round(COALESCE(avg((c.action = 'confirmed')::int), 0), 2) AS confirmation_rate
FROM Signups s
LEFT JOIN Confirmations c ON c.user_id = s.user_id
GROUP BY s.user_id;
```

SQL 50 — Basic aggregate functions

Q135. Not Boring Movies · LeetCode 620 · Easy

`WHERE id % 2 = 1 AND description <> 'boring' ORDER BY rating DESC`. → *Solved in the book:* Ch 18 — P2 · Odd-numbered, not-boring movies

Q136. ⋆ Average Selling Price · LeetCode 1251 · Easy

Range join each sale to the price valid on its date (`purchase_date BETWEEN start_date AND end_date`), then revenue ÷ units; `LEFT JOIN` from `Prices` and `COALESCE(…, 0)` for products never sold. → *Solved in the book:* Ch 18 — P37 · Average selling price

Q137. Project Employees I · LeetCode 1075 · Easy

Join and average per project; `avg` of an integer is `numeric` in PostgreSQL, so `round(…, 2)` works directly.

```sql
SELECT p.project_id, round(avg(e.experience_years), 2) AS average_years
FROM Project p
JOIN Employee e ON e.employee_id = p.employee_id
GROUP BY p.project_id;
```

Q138. Percentage of Users Attended a Contest · LeetCode 1633 · Easy

Registrations per contest × 100 ÷ total users (a scalar subquery), rounded, sorted by percentage desc then `contest_id`. → *Solved in the book:* Ch 19 — 19.9.3 Share of members who borrowed each book

Q139. Queries Quality and Percentage · LeetCode 1211 · Easy

`avg(rating::numeric / position)` for quality and `100.0 * avg((rating < 3)::int)` for the poor percentage, both rounded to 2. LeetCode's test data includes rows with a NULL `query_name`, which the expected output leaves out, so add `WHERE query_name IS NOT NULL`. → *Solved in the book:* Ch 18 — P34 · Search query quality

Q140. ⋆ Monthly Transactions I · LeetCode 1193 · Medium

Group by month and country; conditional aggregation with `FILTER` gives the approved counts and amounts.

```sql
SELECT to_char(trans_date, 'YYYY-MM') AS month, country,
       count(*)                                          AS trans_count,
       count(*) FILTER (WHERE state = 'approved')        AS approved_count,
       sum(amount)                                       AS trans_total_amount,
       COALESCE(sum(amount) FILTER (WHERE state = 'approved'), 0) AS approved_total_amount
FROM Transactions
GROUP BY 1, 2;
```

Q141. ⋆ Immediate Food Delivery II · LeetCode 1174 · Medium

Keep each customer's first order (`DISTINCT ON` or `min(order_date)`), then the percentage where order date = preferred date. → *Solved in the book:* Ch 19 — 19.8.4 Same-day first bookings

Q142. ⋆ Game Play Analysis IV · LeetCode 550 · Medium

Find each player's first login, check whether a row exists for the next day, and divide by the number of players (rounded to 2). → *Solved in the book:* Ch 18 — P38 · Came back the next day

SQL 50 — Sorting and grouping

Q143. Number of Unique Subjects Taught by Each Teacher · LeetCode 2356 · Easy

The same subject can appear in several departments, so count distinct subjects.

```sql
SELECT teacher_id, count(DISTINCT subject_id) AS cnt
FROM Teacher
GROUP BY teacher_id;
```

Q144. User Activity for the Past 30 Days I · LeetCode 1141 · Easy

The 30 days ending 2019-07-27 *inclusive* start on 2019-06-28; count distinct users per day.

```sql
SELECT activity_date AS day, count(DISTINCT user_id) AS active_users
FROM Activity
WHERE activity_date BETWEEN DATE '2019-07-27' - 29 AND DATE '2019-07-27'
GROUP BY activity_date;
```

Q145. ⋆ Product Sales Analysis III · LeetCode 1070 · Medium

Find each product's first year, then return **all** sales rows from that year (several rows can tie), e.g. `WHERE (product_id, year) IN (SELECT product_id, min(year) … GROUP BY product_id)` or `rank() = 1` . → *Solved in the book:* Ch 19 — 19.3.8 First year on loan (keep the ties)

Q146. Classes More Than 5 Students · LeetCode 596 · Easy

Group by class and keep classes with at least 5 students (`(student, class)` is the key, so no `DISTINCT` is needed).

```sql
SELECT class FROM Courses GROUP BY class HAVING count(*) >= 5;
```

Q147. Find Followers Count · LeetCode 1729 · Easy

`GROUP BY user_id` , `count(follower_id)` , `ORDER BY user_id` . → *Solved in the book:* Ch 18 — P12 · Follower count

Q148. Biggest Single Number · LeetCode 619 · Easy

Keep numbers that occur once (`GROUP BY num HAVING count(*) = 1`), then take `max(num)` in an outer query, which also returns NULL when there is none. → *Solved in the book:* Ch 18 — P9 · Largest number that appears once

Q149. ⋆ Customers Who Bought All Products · LeetCode 1045 · Medium

Relational division: a customer qualifies when their count of *distinct* products equals the number of products, `HAVING count(DISTINCT product_key) = (SELECT count(*) FROM Product)` . → *Solved in the book:* Ch 19 — 19.7.6 Students enrolled in every course

SQL 50 — Advanced select and joins

Q150. The Number of Employees Which Report to Each Employee · LeetCode 1731 · Easy

Self-join manager to reports, count and average the reports' ages, rounded to an integer.

```sql
SELECT m.employee_id, m.name, count(*) AS reports_count, round(avg(e.age)) AS average_age
FROM Employees m
JOIN Employees e ON e.reports_to = m.employee_id
GROUP BY m.employee_id, m.name
ORDER BY m.employee_id;
```

Q151. Primary Department for Each Employee · LeetCode 1789 · Easy

Keep the row flagged `'Y'`, or the only row of employees with one department: `WHERE primary_flag = 'Y' OR employee_id IN (… HAVING count(*) = 1)`. → *Solved in the book:* Ch 19 — 19.3.3 One home office per employee

Q152. Triangle Judgement · LeetCode 610 · Easy

`CASE WHEN x + y > z AND x + z > y AND y + z > x THEN 'Yes' ELSE 'No' END AS triangle`. → *Solved in the book:* Ch 18 — P8 · Triangle test

Q153. ⋆ Consecutive Numbers · LeetCode 180 · Medium

Compare each row with the next two by id order using `lead(num, 1)` and `lead(num, 2)` and return `DISTINCT num` where all three match. Ids can have gaps, so use window order, not `id + 1`. → *Solved in the book:* Ch 19 — 19.5.2 Values repeated three times in a row

Q154. ⋆ Product Price at a Given Date · LeetCode 1164 · Medium

Latest change on or before 2019-08-16 per product (`DISTINCT ON (product_id) … ORDER BY product_id, change_date DESC`), and price 10 for products with no change by then. → *Solved in the book:* Ch 18 — P33 · Price on a given date

Q155. ⋆ Last Person to Fit in the Bus · LeetCode 1204 · Medium

Running total of weight in `turn` order; return the person with the largest turn whose running total is ≤ 1000. → *Solved in the book:* Ch 19 — 19.4.2 Last person who fits in the lift

Q156. Count Salary Categories · LeetCode 1907 · Medium

All three categories must appear even with 0 accounts, so start from a fixed list of categories and `LEFT JOIN` the classified accounts.

```sql
SELECT c.category, count(a.account_id) AS accounts_count
FROM (VALUES ('Low Salary', 1), ('Average Salary', 2), ('High Salary', 3)) AS c(category, k)
LEFT JOIN Accounts a
       ON c.k = CASE WHEN a.income < 20000 THEN 1 WHEN a.income <= 50000 THEN 2 ELSE 3 END
GROUP BY c.category, c.k;
```

SQL 50 — Subqueries

Q157. Employees Whose Manager Left the Company · LeetCode 1978 · Easy

Salary below 30000, a non-NULL manager, and no row for that manager any more (anti-join).

```sql
SELECT e.employee_id
FROM Employees e
WHERE e.salary < 30000 AND e.manager_id IS NOT NULL
  AND NOT EXISTS (SELECT 1 FROM Employees m WHERE m.employee_id = e.manager_id)
ORDER BY e.employee_id;
```

Q158. ⋆ Exchange Seats · LeetCode 626 · Medium

Odd ids take the next student, even ids the previous one; the last odd id keeps its own student when there is no next row.

```sql
SELECT id,
       CASE WHEN id % 2 = 1 THEN COALESCE(lead(student) OVER (ORDER BY id), student)
            ELSE lag(student) OVER (ORDER BY id) END AS student
FROM Seat
ORDER BY id;
```

Q159. Movie Rating · LeetCode 1341 · Medium

Two independent top-1 queries (most ratings by user; best February 2020 average by movie), each breaking ties alphabetically, stacked with `UNION ALL` (not `UNION`: a user name can equal a movie title). Each branch needs parentheses to have its own `ORDER BY` / `LIMIT`.

```sql
(SELECT u.name AS results
 FROM MovieRating r JOIN Users u ON u.user_id = r.user_id
 GROUP BY u.user_id, u.name ORDER BY count(*) DESC, u.name LIMIT 1)
UNION ALL
(SELECT m.title
 FROM MovieRating r JOIN Movies m ON m.movie_id = r.movie_id
 WHERE r.created_at >= DATE '2020-02-01' AND r.created_at < DATE '2020-03-01'
 GROUP BY m.movie_id, m.title ORDER BY avg(r.rating) DESC, m.title LIMIT 1);
```

Q160. ⋆ Restaurant Growth · LeetCode 1321 · Medium

Total per day first (several customers per day), then a 7-day moving sum with a `RANGE` frame of 6 days, and keep only days that have a full 7-day window.

```sql
WITH daily AS (SELECT visited_on, sum(amount) AS amount FROM Customer GROUP BY visited_on),
     w AS (SELECT visited_on,
                  sum(amount) OVER (ORDER BY visited_on
                                    RANGE BETWEEN INTERVAL '6 days' PRECEDING AND CURRENT ROW) AS amount,
                  min(visited_on) OVER () AS first_day
           FROM daily)
SELECT visited_on, amount, round(amount / 7.0, 2) AS average_amount
FROM w WHERE visited_on >= first_day + 6
ORDER BY visited_on;
```

Q161. ⋆ Friend Requests II: Who Has the Most Friends · LeetCode 602 · Medium

A friendship counts for both people, so stack requester ids and accepter ids with `UNION ALL`, count per id and take the top one. (The follow-up "return all ties" uses `rank() = 1`, as in the book's version.)

```sql
WITH ids AS (SELECT requester_id AS id FROM RequestAccepted
             UNION ALL
             SELECT accepter_id FROM RequestAccepted)
SELECT id, count(*) AS num FROM ids GROUP BY id ORDER BY num DESC LIMIT 1;
```

→ *Ties variant in the book:* Ch 19 — 19.3.9 Busiest collaborators (all ties)

Q162. Investments in 2016 · LeetCode 585 · Medium

Two window counts per row: rows sharing `tiv_2015` (must be > 1) and rows sharing `(lat, lon)` (must be 1); sum `tiv_2016` of the survivors. The columns are floats, so write `round(sum(tiv_2016)::numeric, 2)`. → *Solved in the book:* Ch 19 — 19.7.7 Shop revenue under two uniqueness rules

Q163. ⋆ Department Top Three Salaries · LeetCode 185 · Hard

`dense_rank() OVER (PARTITION BY departmentId ORDER BY salary DESC)` in a subquery, keep rank ≤ 3 (top three *distinct* salaries, ties included), join the department name. → *Solved in the book:* Ch 19 — 19.3.2 Top three distinct scores per subject

SQL 50 — Advanced string functions, regex and clauses

Q164. Fix Names in a Table · LeetCode 1667 · Easy

Upper-case the first letter, lower-case the rest. (`initcap` would also capitalise letters after spaces or hyphens.)

```sql
SELECT user_id, upper(left(name, 1)) || lower(substring(name FROM 2)) AS name
FROM Users
ORDER BY user_id;
```

Q165. Patients With a Condition · LeetCode 1527 · Easy

A code starting with `DIAB1` is at the start of the string or after a space; a plain `LIKE '%DIAB1%'` would wrongly match `ACNE +DIAB100`.

```sql
SELECT patient_id, patient_name, conditions
FROM Patients
WHERE conditions ~ '(^| )DIAB1';
```

Q166. ★ Delete Duplicate Emails · LeetCode 196 · Easy

A `DELETE` (not a `SELECT`) that removes every row for which a row with the same e-mail and a smaller id exists.

```sql
DELETE FROM Person p
USING Person q
WHERE p.email = q.email AND p.id > q.id;
```

Q167. ★ Second Highest Salary · LeetCode 176 · Medium

Second distinct value with `OFFSET 1`, wrapped in a scalar subquery so that "no second salary" returns one row with NULL instead of no rows: `SELECT (SELECT DISTINCT salary FROM Employee ORDER BY salary DESC OFFSET 1 LIMIT 1) AS "SecondHighestSalary"`. → *Solved in the book:* Ch 18 — P39 · Second-highest salary · Ch 19 — 19.3.1 Second-fastest lap time

Q168. ★ Group Sold Products By The Date · LeetCode 1484 · Easy

Per date: `count(DISTINCT product)` and `string_agg(DISTINCT product, ',' ORDER BY product)`. → *Solved in the book:* Ch 18 — P15 · Products sold per day · Ch 19 — 19.8.3 Titles borrowed per day

Q169. List the Products Ordered in a Period · LeetCode 1327 · Easy

Filter to February 2020 with a half-open range, total per product, keep totals ≥ 100.

```sql
SELECT p.product_name, sum(o.unit) AS unit
FROM Products p
JOIN Orders o ON o.product_id = p.product_id
WHERE o.order_date >= DATE '2020-02-01' AND o.order_date < DATE '2020-03-01'
GROUP BY p.product_id, p.product_name
HAVING sum(o.unit) >= 100;
```

Q170. Find Users With Valid E-Mails · LeetCode 1517 · Easy

Anchor a regular expression at both ends and escape the dot: `WHERE mail ~ '^[A-Za-z][A-Za-z0-9_.-]*@leetcode[.]com$'` (`~` is case-sensitive, so `@LEETCODE.COM` is rejected). → *Solved in the book:* Ch 18 — P7 · Valid company e-mails

Most-asked classics beyond SQL 50

Q171. ★ Combine Two Tables · LeetCode 175 · Easy

Every person must appear, with NULL city and state when there is no address: a `LEFT JOIN`.

```sql
SELECT p.firstName, p.lastName, a.city, a.state
FROM Person p
LEFT JOIN Address a ON a.personId = p.personId;
```

Q172. ★ Nth Highest Salary · LeetCode 177 · Medium

Inside a function, rank distinct salaries with `dense_rank` and pick rank N, wrapped in a scalar subquery so a missing Nth salary (or N < 1) gives NULL. Qualify columns (`e.salary`), because the output column `Salary` is also a PL/pgSQL variable and an unqualified `salary` is ambiguous.

```sql
CREATE OR REPLACE FUNCTION NthHighestSalary(N INT) RETURNS TABLE (Salary INT) AS $$
BEGIN
  RETURN QUERY
  SELECT (SELECT DISTINCT r.salary
          FROM (SELECT e.salary, dense_rank() OVER (ORDER BY e.salary DESC) AS dr
                FROM Employee e) r
          WHERE r.dr = N);
END;
$$ LANGUAGE plpgsql;
```

Q173. ★ Rank Scores · LeetCode 178 · Medium

`dense_rank()` gives consecutive ranks without gaps after ties.

```sql
SELECT score, dense_rank() OVER (ORDER BY score DESC) AS "rank"
FROM Scores
ORDER BY score DESC;
```

Q174. ★ Employees Earning More Than Their Managers · LeetCode 181 · Easy

Self-join each employee to their manager's row and compare salaries.

```sql
SELECT e.name AS "Employee"
FROM Employee e
JOIN Employee m ON m.id = e.managerId
WHERE e.salary > m.salary;
```

Q175. ★ Duplicate Emails · LeetCode 182 · Easy

Group by e-mail and keep groups with more than one row.

```sql
SELECT email AS "Email" FROM Person GROUP BY email HAVING count(*) > 1;
```

Q176. ★ Customers Who Never Order · LeetCode 183 · Easy

Anti-join. `NOT EXISTS` is safe even if `customerId` had NULLs, unlike `NOT IN`.

```sql
SELECT c.name AS "Customers"
FROM Customers c
WHERE NOT EXISTS (SELECT 1 FROM Orders o WHERE o.customerId = c.id);
```

Q177. ★ Department Highest Salary · LeetCode 184 · Medium

`rank() = 1` per department keeps everyone tied for the top salary.

```sql
SELECT d.name AS "Department", e.name AS "Employee", e.salary AS "Salary"
FROM (SELECT *, rank() OVER (PARTITION BY departmentId ORDER BY salary DESC) AS r
      FROM Employee) e
JOIN Department d ON d.id = e.departmentId
WHERE e.r = 1;
```

Q178. ★ Trips and Users · LeetCode 262 · Hard

Join trips to users twice (client and driver), keep only trips where both are unbanned and the date is 2013-10-01 to 2013-10-03, then per day `round(avg((status <> 'completed')::int), 2)` as the cancellation rate. → *Solved in the book:* Ch 19 — 19.9.1 Ride cancellation rate (the "trips and users" classic)

Q179. ★ Human Traffic of Stadium · LeetCode 601 · Hard

Keep rows with `people >= 100`, group consecutive ids with `id - row_number()` (gaps and islands), and return rows whose island has 3 or more rows, ordered by `visit_date`. → *Solved in the book:* Ch 19 — 19.5.1 Runs of busy days at the gate

Q180. ★ Tree Node · LeetCode 608 · Medium

Root has no parent; Inner has at least one child; everything else is a Leaf. Check "has a child" with `EXISTS` (a `NOT IN` on `p_id` would hit the NULL of the root).

```sql
SELECT id,
       CASE WHEN p_id IS NULL THEN 'Root'
            WHEN EXISTS (SELECT 1 FROM Tree c WHERE c.p_id = t.id) THEN 'Inner'
            ELSE 'Leaf' END AS type
FROM Tree t;
```

Q181. Swap Salary · LeetCode 627 · Easy

One `UPDATE` with a `CASE`; no temporary table is needed, because every row is computed from its old value.

```sql
UPDATE Salary SET sex = CASE sex WHEN 'm' THEN 'f' ELSE 'm' END;
```

Q182. Customer Placing the Largest Number of Orders · LeetCode 586 · Easy

Group by customer, order by the count and take one row (use `rank() = 1` for the follow-up "several customers tie").

```sql
SELECT customer_number
FROM Orders
GROUP BY customer_number
ORDER BY count(*) DESC
LIMIT 1;
```

Q183. Count Student Number in Departments · LeetCode 580 · Medium

Start from departments so empty ones show 0, and count a student column, not `*`.

```sql
SELECT d.dept_name, count(s.student_id) AS student_number
FROM Department d
LEFT JOIN Student s ON s.dept_id = d.dept_id
GROUP BY d.dept_id, d.dept_name
ORDER BY student_number DESC, d.dept_name;
```

Game Play Analysis series

Q184. ⋆ Game Play Analysis I · LeetCode 511 · Easy

`SELECT player_id, min(event_date) AS first_login FROM Activity GROUP BY player_id`. → *Solved in the book:* Ch 18 — P11 · First login per player

Q185. Game Play Analysis II · LeetCode 512 · Easy

First row per player: `DISTINCT ON (player_id) player_id, device_id … ORDER BY player_id, event_date`. → *Solved in the book:* Ch 18 — P40 · Device used at first login

Q186. ⋆ Game Play Analysis III · LeetCode 534 · Medium

Running total: `sum(games_played) OVER (PARTITION BY player_id ORDER BY event_date)`. → *Solved in the book:* Ch 18 — P41 · Games played so far

Q187. Game Play Analysis V · LeetCode 1097 · Hard

Install date = first login; per install date, count installs and the share who also played on install date + 1 (day-1 retention), rounded to 2. → *Solved in the book:* Ch 19 — 19.8.1 Day-1 retention

(Game Play Analysis IV is Q142.)

Ranking, top-N and medians

Q188. ⋆ Median Employee Salary · LeetCode 569 · Hard

Number rows by salary within each company and count them; the median rows are those whose position is between n/2 and n/2 + 1 (one row for odd n, two for even n).

```sql
SELECT id, company, salary
FROM (SELECT *, row_number() OVER (PARTITION BY company ORDER BY salary, id) AS rn,
              count(*)     OVER (PARTITION BY company)                    AS n
      FROM Employee) t
WHERE rn BETWEEN n / 2.0 AND n / 2.0 + 1;
```

Q189. ⋆ Find Median Given Frequency of Numbers · LeetCode 571 · Hard

A cumulative sum of frequencies gives each number's range of positions; a number is a median value when its range covers the middle position (total/2). Average those numbers.

```sql
WITH c AS (SELECT num, frequency,
                  sum(frequency) OVER (ORDER BY num) AS upto,
                  sum(frequency) OVER ()             AS total
           FROM Numbers)
SELECT round(avg(num)::numeric, 1) AS median
FROM c
WHERE upto >= total / 2.0 AND upto - frequency <= total / 2.0;
```

For a plain column, PostgreSQL's `percentile_cont(0.5) WITHIN GROUP (ORDER BY x)` computes the median directly.

Q190. Project Employees III · LeetCode 1077 · Medium

Most experienced employee(s) per project, ties kept: `rank() = 1`.

```
SELECT project_id, employee_id
FROM (SELECT p.project_id, p.employee_id,
             rank() OVER (PARTITION BY p.project_id ORDER BY e.experience_years DESC) AS r
      FROM Project p JOIN Employee e ON e.employee_id = p.employee_id) t
WHERE r = 1;
```

Q191. Sales Analysis I · LeetCode 1082 · Easy

Total per seller, then every seller tied for the maximum. A window function can rank aggregated values in the same query level.

```
SELECT seller_id
FROM (SELECT seller_id, rank() OVER (ORDER BY sum(price) DESC) AS r
      FROM Sales GROUP BY seller_id) t
WHERE r = 1;
```

Q192. ⋆ Highest Grade For Each Student · LeetCode 1112 · Medium

First row per student by grade descending, then the smallest course id.

```
SELECT DISTINCT ON (student_id) student_id, course_id, grade
FROM Enrollments
ORDER BY student_id, grade DESC, course_id;
```

Q193. Tournament Winners · LeetCode 1194 · Hard

Unpivot matches into (player, points) rows with `UNION ALL`, total per player, then the first player per group by total desc and `player_id` asc.

```
WITH pts AS (SELECT first_player AS player_id, first_score AS score FROM Matches
             UNION ALL
             SELECT second_player, second_score FROM Matches)
SELECT DISTINCT ON (p.group_id) p.group_id, p.player_id
FROM Players p
LEFT JOIN pts ON pts.player_id = p.player_id
GROUP BY p.group_id, p.player_id
ORDER BY p.group_id, COALESCE(sum(pts.score), 0) DESC, p.player_id;
```

Q194. Get the Second Most Recent Activity · LeetCode 1369 · Hard

Number each user's activities newest first and count them; keep row 2, or the only row when a user has one activity.

```
SELECT username, activity, startDate AS "startDate", endDate AS "endDate"
FROM (SELECT *, row_number() OVER (PARTITION BY username ORDER BY startDate DESC) AS rn,
                count(*)     OVER (PARTITION BY username)                          AS n
      FROM UserActivity) t
WHERE rn = 2 OR n = 1;
```

Q195. Average Salary: Departments VS Company · LeetCode 615 · Hard

Attach the company-wide monthly average to every salary row with a window, then group by month and department and compare.

```sql
WITH t AS (SELECT to_char(s.pay_date, 'YYYY-MM') AS pay_month, e.department_id, s.amount,
                  avg(s.amount) OVER (PARTITION BY to_char(s.pay_date, 'YYYY-MM')) AS company_avg
           FROM Salary s JOIN Employee e ON e.employee_id = s.employee_id)
SELECT pay_month, department_id,
       CASE WHEN avg(amount) > max(company_avg) THEN 'higher'
            WHEN avg(amount) < max(company_avg) THEN 'lower' ELSE 'same' END AS comparison
FROM t
GROUP BY pay_month, department_id;
```

Q196. Find Cumulative Salary of an Employee · LeetCode 579 · Hard

A `RANGE BETWEEN 2 PRECEDING AND CURRENT ROW` frame on the month number sums this month and the two calendar months before it (a missing month simply contributes nothing); then drop each employee's latest month.

```sql
SELECT id, month, "Salary"
FROM (SELECT id, month,
             sum(salary) OVER (PARTITION BY id ORDER BY month
                               RANGE BETWEEN 2 PRECEDING AND CURRENT ROW) AS "Salary",
             max(month) OVER (PARTITION BY id) AS last_month
      FROM Employee) t
WHERE month < last_month
ORDER BY id, month DESC;
```

Q197. The Most Recent Three Orders · LeetCode 1532 · Medium

Top-N per group: `row_number()` per customer by date descending, keep ≤ 3.

```sql
SELECT customer_name, customer_id, order_id, order_date
FROM (SELECT c.name AS customer_name, c.customer_id, o.order_id, o.order_date,
             row_number() OVER (PARTITION BY c.customer_id ORDER BY o.order_date DESC) AS rn
      FROM Customers c JOIN Orders o ON o.customer_id = c.customer_id) t
WHERE rn <= 3
ORDER BY customer_name, customer_id, order_date DESC;
```

Q198. Find the Quiet Students in All Exams · LeetCode 1412 · Hard

Rank each exam's scores both ascending and descending; a student is quiet if they took an exam and never had rank 1 in either direction. → *Solved in the book:* Ch 19 — 19.3.6 Pupils who are never top or bottom

Consecutive rows, streaks and ranges

Q199. ⋆ Report Contiguous Dates · LeetCode 1225 · Hard

Stack failed and succeeded days with their state, keep 2019, and apply gaps and islands per state (`date - row_number()`); each island becomes one period.

```sql
WITH days AS (SELECT fail_date AS d, 'failed' AS period_state FROM Failed
              UNION ALL
              SELECT success_date, 'succeeded' FROM Succeeded),
     g AS (SELECT d, period_state,
                  d - (row_number() OVER (PARTITION BY period_state ORDER BY d))::int AS grp
           FROM days
           WHERE d BETWEEN DATE '2019-01-01' AND DATE '2019-12-31')
SELECT period_state, min(d) AS start_date, max(d) AS end_date
FROM g
GROUP BY period_state, grp
ORDER BY start_date;
```

Q200. ⋆ Find the Start and End Number of Continuous Ranges · LeetCode 1285 · Medium

In a run of consecutive integers, `log_id - row_number()` is constant, so group by it.

```sql
SELECT min(log_id) AS start_id, max(log_id) AS end_id
FROM (SELECT log_id, log_id - row_number() OVER (ORDER BY log_id) AS grp FROM Logs) t
GROUP BY grp
ORDER BY start_id;
```

Q201. Consecutive Available Seats · LeetCode 603 · Easy

A free seat qualifies when the seat right before or after it is also free. The `::int = 1` cast works whether `free` is a boolean or a 0/1 integer.

```sql
SELECT DISTINCT a.seat_id
FROM Cinema a
JOIN Cinema b ON abs(a.seat_id - b.seat_id) = 1
WHERE a.free::int = 1 AND b.free::int = 1
ORDER BY a.seat_id;
```

Q202. ⋆ Active Users · LeetCode 1454 · Medium

De-duplicate to one row per user per day, then gaps and islands on the dates (`login_date - row_number()`) and keep islands of 5 or more days. → *Solved in the book:* Ch 19 — 19.5.3 Members active five days in a row

Q203. Longest Winning Streak · LeetCode 2173 · Hard

Mark each non-win as the start of a new group with a running count of losses/draws, count wins per group, take each player's maximum, and `COALESCE` to 0 for players who never won. → *Solved in the book:* Ch 19 — 19.5.5 Longest winning streak

Q204. Suspicious Bank Accounts · LeetCode 1843 · Medium

Total deposits per account per month, keep months over the limit, then check whether some month and the next calendar month both qualify (self-join on `month + interval '1 month'` or `lag`). → *Solved in the book:* Ch 19 — 19.5.6 Deposits over the limit two months running

Q205. Find Interview Candidates · LeetCode 1811 · Medium

Unpivot each contest into (contest_id, user) medal rows; a user qualifies with 3 golds overall or medals in 3 consecutive contest ids (gaps and islands on `contest_id`). → *Solved in the book:* Ch 19 — 19.5.4 Interview candidates from contest medals

Joins, anti-joins and hierarchies

Q206. ⋆ Market Analysis I · LeetCode 1158 · Medium

Put the 2019 date filter in the `ON` clause of the `LEFT JOIN` ; in `WHERE` it would drop users with no 2019 orders.

```sql
SELECT u.user_id AS buyer_id, u.join_date, count(o.order_id) AS orders_in_2019
FROM Users u
LEFT JOIN Orders o ON o.buyer_id = u.user_id
                  AND o.order_date >= DATE '2019-01-01' AND o.order_date < DATE '2020-01-01'
GROUP BY u.user_id, u.join_date;
```

Q207. Top Travellers · LeetCode 1407 · Easy

`LEFT JOIN` rides, total with `COALESCE(…, 0)` , and group by the user id (two users can share a name).

```sql
SELECT u.name, COALESCE(sum(r.distance), 0) AS travelled_distance
FROM Users u
LEFT JOIN Rides r ON r.user_id = u.id
GROUP BY u.id, u.name
ORDER BY travelled_distance DESC, u.name;
```

Q208. Employees With Missing Information · LeetCode 1965 · Easy

`FULL JOIN` both tables and keep ids missing a name or a salary; with `USING` , `employee_id` is the merged non-NULL key.

```sql
SELECT employee_id
FROM Employees FULL JOIN Salaries USING (employee_id)
WHERE name IS NULL OR salary IS NULL
ORDER BY employee_id;
```

Q209. Students With Invalid Departments · LeetCode 1350 · Easy

Anti-join students to departments.

```sql
SELECT s.id, s.name
FROM Students s
WHERE NOT EXISTS (SELECT 1 FROM Departments d WHERE d.id = s.department_id);
```

Q210. Sales Analysis II · LeetCode 1083 · Easy

Per buyer, `bool_or` asks "did any of this buyer's rows match?": bought S8 and never an iPhone.

```sql
SELECT s.buyer_id
FROM Sales s JOIN Product p ON p.product_id = s.product_id
GROUP BY s.buyer_id
HAVING bool_or(p.product_name = 'S8') AND NOT bool_or(p.product_name = 'iPhone');
```

Q211. ⋆ Sales Analysis III · LeetCode 1084 · Easy

"Sold only in Q1 2019" means the earliest and latest sale dates are both inside the quarter.

```sql
SELECT p.product_id, p.product_name
FROM Product p JOIN Sales s ON s.product_id = p.product_id
GROUP BY p.product_id, p.product_name
HAVING min(s.sale_date) >= DATE '2019-01-01' AND max(s.sale_date) <= DATE '2019-03-31';
```

Q212. Customers Who Bought Products A and B but Not C · LeetCode 1398 · Medium

Same "has / has not" pattern with one `bool_or` per product.

```sql
SELECT c.customer_id, c.customer_name
FROM Customers c JOIN Orders o ON o.customer_id = c.customer_id
GROUP BY c.customer_id, c.customer_name
HAVING bool_or(o.product_name = 'A') AND bool_or(o.product_name = 'B')
   AND NOT bool_or(o.product_name = 'C')
ORDER BY c.customer_id;
```

Q213. All People Report to the Given Manager · LeetCode 1270 · Medium

Walk down from the head (id 1, who is their own manager) with a recursive CTE; excluding id 1 in the anchor stops the self-loop. (The problem caps the depth at 3, so three self-joins also work.)

```sql
WITH RECURSIVE r AS (
  SELECT employee_id FROM Employees WHERE manager_id = 1 AND employee_id <> 1
  UNION
  SELECT e.employee_id FROM Employees e JOIN r ON e.manager_id = r.employee_id
)
SELECT employee_id FROM r;
```

Q214. Find the Subtasks That Did Not Execute · LeetCode 1767 · Hard

Generate every expected subtask with `generate_series` per task, then anti-join the executed ones.

```sql
SELECT t.task_id, s.subtask_id
FROM Tasks t
CROSS JOIN LATERAL generate_series(1, t.subtasks_count) AS s(subtask_id)
WHERE NOT EXISTS (SELECT 1 FROM Executed e
                  WHERE e.task_id = t.task_id AND e.subtask_id = s.subtask_id);
```

Q215. Second Degree Follower · LeetCode 614 · Medium

A second-degree follower both follows someone and is followed by someone: keep followees who also appear as followers, and count their followers. → *Solved in the book:* Ch 18 — P22 · "Middle" users in a follow graph

Q216. ⋆ Sellers With No Sales · LeetCode 1607 · Easy

Anti-join sellers to 2020 orders (`NOT EXISTS` with a half-open date range), ordered by name. → *Solved in the book:* Ch 18 — P23 · Sellers with no 2020 sales · Ch 18 — P24 · Dormant accounts

Q217. Ad-Free Sessions · LeetCode 1809 · Easy

Anti-join on a range: no ad for the same customer with `timestamp BETWEEN start_time AND end_time`. → *Solved in the book:* Ch 18 — P25 · Sessions with no ads

Q218. ⋆ Sales Person · LeetCode 607 · Easy

Sales people with no order for the company named `'RED'`: `NOT EXISTS` over orders joined to company. → *Solved in the book:* Ch 18 — P26 · Reps who never sold to RED

Q219. Winning Candidate · LeetCode 574 · Medium

Count votes per candidate, order by the count descending, `LIMIT 1`, and join the name. → *Solved in the book:* Ch 18 — P21 · Election winner

Q220. Page Recommendations · LeetCode 1264 · Medium

Friends of user 1 come from both columns of `Friendship`; suggest their liked pages except pages user 1 already likes, `DISTINCT`. → *Solved in the book:* Ch 19 — 19.7.1 Page suggestions from friends' likes

Aggregation, rates and reports

Q221. Actors and Directors Who Cooperated At Least Three Times · LeetCode 1050 · Easy

```sql
SELECT actor_id, director_id
FROM ActorDirector
GROUP BY actor_id, director_id
HAVING count(*) >= 3;
```

Q222. Capital Gain/Loss · LeetCode 1393 · Medium

Sells add the price, buys subtract it; a signed `CASE` inside `sum`.

```sql
SELECT stock_name,
       sum(CASE operation WHEN 'Sell' THEN price ELSE -price END) AS capital_gain_loss
FROM Stocks
GROUP BY stock_name;
```

Q223. Bank Account Summary II · LeetCode 1587 · Easy

```sql
SELECT u.name, sum(t.amount) AS balance
FROM Users u JOIN Transactions t ON t.account = u.account
GROUP BY u.account, u.name
HAVING sum(t.amount) > 10000;
```

Q224. Daily Leads and Partners · LeetCode 1693 · Easy

```sql
SELECT date_id, make_name,
       count(DISTINCT lead_id) AS unique_leads, count(DISTINCT partner_id) AS unique_partners
FROM DailySales
GROUP BY date_id, make_name;
```

Q225. Find Total Time Spent by Each Employee · LeetCode 1741 · Easy

```sql
SELECT event_day AS day, emp_id, sum(out_time - in_time) AS total_time
FROM Employees
GROUP BY event_day, emp_id;
```

Q226. The Latest Login in 2020 · LeetCode 1890 · Easy

A half-open range on the timestamp (`< 2021-01-01`) keeps logins late on 31 December, which `<= '2020-12-31'` would miss.

```sql
SELECT user_id, max(time_stamp) AS last_stamp
FROM Logins
WHERE time_stamp >= '2020-01-01' AND time_stamp < '2021-01-01'
GROUP BY user_id;
```

Q227. Calculate Special Bonus · LeetCode 1873 · Easy

```sql
SELECT employee_id,
       CASE WHEN employee_id % 2 = 1 AND name NOT LIKE 'M%' THEN salary ELSE 0 END AS bonus
FROM Employees
ORDER BY employee_id;
```

Q228. Customer Order Frequency · LeetCode 1511 · Easy

Two conditional sums, one per month, both required in `HAVING`.

```sql
SELECT c.customer_id, c.name
FROM Customers c
JOIN Orders o  ON o.customer_id = c.customer_id
JOIN Product p ON p.product_id = o.product_id
GROUP BY c.customer_id, c.name
HAVING sum(o.quantity * p.price) FILTER (WHERE o.order_date >= '2020-06-01' AND o.order_date < '2020-07-01') >=
100
   AND sum(o.quantity * p.price) FILTER (WHERE o.order_date >= '2020-07-01' AND o.order_date < '2020-08-01') >=
100;
```

Q229. Warehouse Manager · LeetCode 1571 · Easy

```sql
SELECT w.name AS warehouse_name, sum(w.units * p.width * p.length * p.height) AS volume
FROM Warehouse w JOIN Products p ON p.product_id = w.product_id
GROUP BY w.name;
```

Q230. Number of Calls Between Two Persons · LeetCode 1699 · Medium

Normalise each pair with `least` / `greatest` so (1, 2) and (2, 1) fall into the same group.

```sql
SELECT least(from_id, to_id) AS person1, greatest(from_id, to_id) AS person2,
       count(*) AS call_count, sum(duration) AS total_duration
FROM Calls
GROUP BY 1, 2;
```

Q231. Reported Posts · LeetCode 1113 · Easy

"Yesterday" is 2019-07-04; a post reported twice for the same reason counts once.

```sql
SELECT extra AS report_reason, count(DISTINCT post_id) AS report_count
FROM Actions
WHERE action = 'report' AND action_date = DATE '2019-07-04'
GROUP BY extra;
```

Q232. Immediate Food Delivery I · LeetCode 1173 · Easy

The average of a 0/1 flag is a fraction; × 100 for a percentage.

```sql
SELECT round(100.0 * avg((order_date = customer_pref_delivery_date)::int), 2) AS immediate_percentage
FROM Delivery;
```

Q233. Friend Requests I: Overall Acceptance Rate · LeetCode 597 · Easy

Distinct accepted pairs ÷ distinct requested pairs, 0.00 when there are no requests (`NULLIF` avoids division by zero, `COALESCE` turns the NULL into 0).

```sql
SELECT round(COALESCE(
        (SELECT count(*) FROM (SELECT DISTINCT requester_id, accepter_id FROM RequestAccepted) a)::numeric
        / NULLIF((SELECT count(*) FROM (SELECT DISTINCT sender_id, send_to_id FROM FriendRequest) r), 0),
      0), 2) AS accept_rate;
```

Q234. Get Highest Answer Rate Question · LeetCode 578 · Medium

Answer rate = answers ÷ shows per question; order by it (NULLs last for questions never shown) and take the smallest id on ties.

```sql
SELECT question_id AS survey_log
FROM SurveyLog
GROUP BY question_id
ORDER BY count(*) FILTER (WHERE action = 'answer')::numeric
         / NULLIF(count(*) FILTER (WHERE action = 'show'), 0) DESC NULLS LAST,
         question_id
LIMIT 1;
```

Q235. Monthly Transactions II · LeetCode 1205 · Medium

Stack approved transactions and chargebacks (chargebacks dated by their own date, with the original amount) using `UNION ALL`, then pivot per month and country with conditional sums. → *Solved in the book:* Ch 18 — P28 · Monthly approvals and chargebacks

Q236. Weather Type in Each Country · LeetCode 1294 · Easy

Filter November 2019, average per country, then `CASE` on the average (≤ 15 Cold, ≥ 25 Hot, else Warm). → *Solved in the book:* Ch 18 — P29 · Weather class per country in November 2019

Q237. Ads Performance · LeetCode 1322 · Easy

CTR = clicks ÷ (clicks + views) × 100, and 0 when both are zero (`COALESCE(… / NULLIF(…, 0), 0)`), sorted by CTR desc then `ad_id`. → *Solved in the book:* Ch 18 — P35 · Ad click-through rate

Q238. Active Businesses · LeetCode 1126 · Medium

Attach each event type's average with `avg(occurences) OVER (PARTITION BY event_type)`, keep rows above it, and return businesses with more than one such row. → *Solved in the book:* Ch 18 — P42 · Active businesses

Q239. Running Total for Different Genders · LeetCode 1308 · Medium

`sum(score_points) OVER (PARTITION BY gender ORDER BY day)`, ordered by gender and day. → *Solved in the book:* Ch 19 — 19.4.1 Running total per team

Q240. Popularity Percentage · LeetCode 2720 · Hard

Count each user's friends from both columns (`UNION ALL` of the two directions), divide by the number of distinct users × 100, round to 2. → *Solved in the book:* Ch 19 — 19.7.2 Popularity percentage

Q241. Team Scores in Football Tournament · LeetCode 1212 · Medium

For every team (`LEFT JOIN` so teams with no matches get 0), sum 3 / 1 / 0 points for wins, draws and losses as host or guest; sort by points desc, then `team_id`. → *Solved in the book:* Ch 19 — 19.7.4 League table

Q242. Number of Transactions per Visit · LeetCode 1336 · Hard

Count transactions per visit (`LEFT JOIN`, so visits with none count 0), build every k from 0 to the maximum with `generate_series`, and count visits per k with a `LEFT JOIN` so empty buckets show 0. → *Solved in the book:* Ch 19 — 19.8.2 Transactions per visit histogram

Pivot, unpivot and string building

Q243. ★ Reformat Department Table · LeetCode 1179 · Easy

Static pivot: one conditional aggregate per month.

```sql
SELECT id,
  sum(revenue) FILTER (WHERE month = 'Jan') AS "Jan_Revenue", sum(revenue) FILTER (WHERE month = 'Feb') AS
"Feb_Revenue",
  sum(revenue) FILTER (WHERE month = 'Mar') AS "Mar_Revenue", sum(revenue) FILTER (WHERE month = 'Apr') AS
"Apr_Revenue",
  sum(revenue) FILTER (WHERE month = 'May') AS "May_Revenue", sum(revenue) FILTER (WHERE month = 'Jun') AS
"Jun_Revenue",
  sum(revenue) FILTER (WHERE month = 'Jul') AS "Jul_Revenue", sum(revenue) FILTER (WHERE month = 'Aug') AS
"Aug_Revenue",
  sum(revenue) FILTER (WHERE month = 'Sep') AS "Sep_Revenue", sum(revenue) FILTER (WHERE month = 'Oct') AS
"Oct_Revenue",
  sum(revenue) FILTER (WHERE month = 'Nov') AS "Nov_Revenue", sum(revenue) FILTER (WHERE month = 'Dec') AS
"Dec_Revenue"
FROM Department
GROUP BY id;
```

Q244. ⋆ Rearrange Products Table · LeetCode 1795 · Easy

Unpivot: a `LATERAL VALUES` list turns the three store columns into rows; drop NULL prices. (`UNION ALL` of three `SELECT`s also works.)

```sql
SELECT p.product_id, v.store, v.price
FROM Products p
CROSS JOIN LATERAL (VALUES ('store1', p.store1), ('store2', p.store2), ('store3', p.store3))
     AS v(store, price)
WHERE v.price IS NOT NULL;
```

Q245. Grand Slam Titles · LeetCode 1783 · Medium

Unpivot the four tournament columns into one column of winner ids, then count per player.

```sql
SELECT p.player_id, p.player_name, count(*) AS grand_slams_count
FROM Championships c
CROSS JOIN LATERAL (VALUES (c.wimbledon), (c.fr_open), (c.us_open), (c.au_open)) AS w(player_id)
JOIN Players p ON p.player_id = w.player_id
GROUP BY p.player_id, p.player_name;
```

Q246. Students Report By Geography · LeetCode 618 · Hard

Number students alphabetically within each continent (`row_number`), then pivot on that number with one `max(name) FILTER (WHERE continent = …)` per continent. → *Solved in the book:* Ch 19 — 19.6.1 Names by region, side by side

Q247. Dynamic Pivoting of a Table · LeetCode 2252 · Hard

The column list is not known in advance, so a procedure reads the distinct stores, builds the pivot query as text with `format('%I')` for identifiers, and runs it with `EXECUTE`. → *Solved in the book:* Ch 19 — 19.6.2 Dynamic pivot: one column per store

Q248. Dynamic Unpivoting of a Table · LeetCode 2253 · Hard

Read the store column names from the catalog (`information_schema.columns`), build one `SELECT … UNION ALL` branch per column with `format`, and `EXECUTE` it. → *Solved in the book:* Ch 19 — 19.6.3 Dynamic unpivot: store columns back to rows

Q249. Evaluate Boolean Expression · LeetCode 1440 · Medium

Join `Variables` twice (left and right operand), then `CASE operator WHEN '<' THEN l.value < r.value … END`, turned into `'true'` / `'false'`. → *Solved in the book:* Ch 19 — 19.10.1 Evaluate stored comparisons

Q250. Build the Equation · LeetCode 2118 · Hard

Format each term (sign, absolute coefficient, `X^power`, `X` or nothing) and join them with `string_agg(term, '' ORDER BY power DESC) || '=0'`. → *Solved in the book:* Ch 19 — 19.10.2 Build a polynomial equation string

Sources of the question lists

Question titles and wording only were taken from these lists; every answer and solution above is written for this book.

- LeetCode — *SQL 50* study plan (leetcode.com/studyplan/top-sql-50) and the LeetCode Database problem set (problem titles and numbers).
- LeetCode Discuss — "SQL Interview Questions Bootcamp" study guide and SQL interview-experience posts.
- GitHub mirrors of the SQL 50 plan: Karthik-S-R/Leetcode-SQL-50, HarshitLaloriya/Leetcode-SQL-50 (section sizes 5/9/8/7/7/7/7), HamzaHassanain/Leetcode_SQL_50_Study_Plan.
- devinterview-io/databases-interview-questions (GitHub) — concept topics: ACID, isolation levels, indexes, normalization.
- "Top 10 frequently asked SQL coding interview questions" (Medium) and "10 questions you must know to ace any SQL interviews" (Towards Data Science).
- Public "Top SQL questions LeetCode" lists (Scribd uploads), used only to confirm which classic problems recur.